

# One GEMM, Many Subgames: A Predictive Cost Model and an End-to-End Wall-Clock Win for Iterated Resolving in Imperfect-Information Games

Max Zhang

Independent Researcher (affiliation placeholder)

June 10, 2026

## Abstract

Train-time iterated resolving — the self-play loop behind DeepStack-, ReBeL-, and Student-of-Games-style agents — has a multiplicative cost: (number of subgame solves)  $\times$  (CFR iterations per solve)  $\times$  (wall-clock per CFR iteration of a solve). Existing levers target the first two factors, sampling variance, or subgame size; the cost *per solve* has been attacked only within a single tree — amortizing it across the population of subgames a resolving sweep generates is, to our knowledge, unclaimed in published work. We observe that iterated resolving structurally generates, at every iteration, a *population* of same-topology depth-limited public-belief-state subgames differing only in entry beliefs. We fuse the population into one level-grouped, structure-of-arrays CFR+ whose per-iteration work is a fixed sequence of batched tensor ops, with equilibrium parity to per-tree solving gated at  $0\text{--}7 \times 10^{-6}$  (float64; CPU training trajectories identical to all recorded digits). A two-regime cost model — speedup near-linear in population size while solves are launch-bound, saturating ( $\sim 2.4\times$ ) once one subgame fills the device, with the saturated plateau calibrated from the microbenchmark itself — transfers from microbenchmark to training loop at both ends.

The transfer numbers are same-substrate mechanism-isolation measurements: the in-loop inner-resolve speedup is measured against the same level-grouped kernel applied per-tree on the same GPU under eager dispatch (a launch-amortized CUDA-Graphs/`torch.compile` sequential arm is a planned, unmeasured ablation). On launch-bound Goofspiel-4 (5 matched seeds):  $11.7\times \rightarrow 11.79\times$ , where the measured 98.4% inner-resolve fraction then closes the total-wall accounting ( $9.99\times$  measured,  $10.09\times$  ceiling; both arms finishing in the same exploitability band). On saturated Goofspiel-5 (236,450 infosets; 3 matched seeds, quiet host):  $2.37\times \rightarrow 2.40\times$ , where total wall improves  $2.00\times$  against its  $2.01\times$  ceiling at the measured 86.1% inner fraction — equal-band certified on CPU by exactness gates and single-solve parity, with the CUDA band test at Goofspiel-5 pending. The practitioner-facing floor comes from our own artifacts: against the strongest non-fused arm we measured — the same kernel on the host CPU — the Goofspiel-4 best-vs-best win is  $\sim 2.7\times$  pending matched CPU timings, and in these runs fusion is what made the GPU the faster substrate in the launch-bound regime (single-run CPU cells). The saturated bound is stated as part of the contribution, not a caveat. The lever is orthogonal in form to existing levers; composition is untested.

## 1 Introduction

Sound depth-limited search with learned value functions is the dominant template for two-player zero-sum imperfect-information games. DeepStack [41], ReBeL [12], and Student of Games [47] all

train by *iterated resolving*: training repeatedly generates public belief states (PBS) — by self-play in ReBeL and Student of Games, by random sampling in DeepStack — re-solves a depth-limited subgame rooted there with a value network supplying leaf continuation values, and uses the solved values as training targets. Naive depth limits without belief-conditioned values are unsound [8], so the inner re-solve is not optional — and it dominates the loop: in our instrumented training runs the inner re-solve accounts for 98.4% of baseline wall-clock at our Goofspiel-4 (G4) operating point (86.1% at Goofspiel-5 (G5), median of 3 quiet-host seeds; §4.4). The inner solver throughout is CFR+ [52, 53, 5], the practical workhorse of the regret-minimization family founded by CFR [57]. The cost of this resolving phase — the bulk of training wall-clock at our operating points — factors as

$$C = \underbrace{N_{\text{solves}}}_{\text{how many subgames}} \times \underbrace{T}_{\text{CFR iterations per solve}} \times \underbrace{c_{\text{solve}}}_{\text{wall-clock per solve-iteration}}. \quad (1)$$

Here  $N_{\text{solves}}$  counts subgame solves over the run,  $T$  is the common per-solve CFR iteration budget, and  $c_{\text{solve}}$  is the average wall-clock per solve-iteration, so Eq. (1) is an accounting identity for the resolving phase; the loop’s remaining phases (network training, measurement) sit outside it and are handled by the Amdahl analysis of §4.2.

**All but one of these levers are actively claimed.** Reducing  $N_{\text{solves}}$  is the territory of TurboReBeL [34], whose primary lever is single-sample multi-iteration data generation — one sampling pass yields training data for all  $T$  CFR iterations — with poker-isomorphism augmentation as a secondary lever; each solve is still executed one subgame at a time. Reducing  $T$  is the regret-minimizer-design line: DCFR/PCFR+ variants [9, 18], meta-learned predictive minimizers trained over a distribution of subgames but deployed on each subgame *sequentially* [50, 51], learned-predictive deep variants [56], and search-discovered update rules [35]. Sample-efficiency levers (Monte Carlo CFR (MCCFR) and its variance-reduced descendants VR-MCCFR, ESCHER, DREAM [30, 46, 38, 49]) reduce the variance of each traversal; subgame-*size* levers learn smaller abstractions [28]. The third factor,  $c_{\text{solve}}$ , has so far only been attacked *within a single tree*: the GPU-CFR line parallelizes one solve’s tree walk on device [24; 25, with the actively developed NoRegret library; a further per-game data point in 4]. The structural neighbors are distinct: batched *normal-form* amortized solvers [37] trade exactness for a learned map over payoff matrices; task-level decomposition of the game into independently (in-parallel) solvable subtasks [33] parallelizes scheduling, not the solve itself; DeepStack batches its *network evaluations*, not its solves. We find no published work claiming population-level fusion of subgame solves on the GPU-CFR line (a time-stamped check is given in §2; the durable argument is the structural distinctions above, not the citation count). The cost-per-solve axis — making each CFR iteration cheaper by amortizing it across many subgames at once, exactly — is unclaimed in published work.

**The structural observation.** Iterated resolving does not produce one subgame at a time; it produces a *population*. At a fixed depth cut, every public state on the frontier roots a depth-limited subgame, and within a game these continuation trees share their topology — they differ only in the *data* flowing through them: each player’s entry beliefs (ranges) and the per-subgame regret and average-strategy state. Two population dimensions must be kept distinct (notation used throughout):  $K$ , the number of public keys on the cut frontier — the **across-solve fusion axis this paper claims**, and the ceiling of the cost model and the measured median speedups (per-seed values can exceed it slightly through measurement noise and asymmetric launch overhead; §4.2) — and  $B$ , the total number of cut instances (private-deal combinations summed over keys). The within-key range dimension — per-key instance counts summing to  $B$  — is the classical public-state range-vector decomposition of CFR [22, 23], used in DeepStack’s own resolver and carried identically by *both* of our timing arms; what fusion adds is the  $K$  axis. This population shape is exactly

what GPUs are starved for. We compile the shared topology once into a flat structure-of-arrays (SoA) layout, level-grouping nodes by (type, acting player, action count); each CFR+ iteration then becomes a fixed sequence of batched gathers, scatters, and GEMM-like contractions (we use “GEMM” loosely, here and in the title; §3.2) over the whole population — forward reach pass, backward counterfactual-value pass, regret and strategy-sum updates — with *no per-node Python recursion and no per-subgame kernel launches*. Crucially this is more than a `vmap` over independent trees: the compiler must recover range-vector semantics across heterogeneous decision, chance, and variable-action structure at compile time, maintain float64 discipline through hundreds of compounding fixed-point iterations, and integrate into a live training loop — and the fused solver is gated to return the same equilibria as per-tree solving, exact in the gated sense (average-strategy parity  $0\text{--}7 \times 10^{-6}$  in float64 across our benchmark games; a CPU unit gate at  $< 10^{-9}$ ; and a CPU determinism check in which fused and sequential training trajectories are identical to all recorded digits).

**A predictive cost model, including its own ceiling.** Why does fusion pay, and when does it stop paying? A controlled sweep over the number  $N$  of co-solved subgames at fixed per-subgame size (the key sweep of §4.1) gives a two-regime answer:  $\text{speedup}(N) \approx \min(N, 1/\varphi)$ , where  $\varphi \in (0, 1]$  is the *fill fraction* — the fraction of the device one subgame’s kernels occupy (the min form’s second argument sets the regime boundary, not the plateau height; §4.1). When individual subgames are small enough that one solve under-utilizes the device (the *launch-bound* regime), fused time is essentially constant in  $N$  (197.6 to 202.4 ms as  $N$  goes 1 to 12 on Goofspiel-4) while sequential time grows linearly, so speedup tracks  $N$  nearly exactly (1.0/2.0/3.96/7.9/11.7 $\times$ ). When a single subgame already saturates the GPU (Goofspiel-5’s deeper trees), fused time grows with  $N$  and the speedup saturates at  $\sim 2.4\times$  — residual launch-overhead recovery. We state the saturated bound as part of the contribution, not as a caveat: the microbenchmark transfers to the in-loop inner-resolve outcome at both ends (§4.2, §4.4; the saturated plateau is calibrated from the microbenchmark itself, §4.1), and a cheap marginal-cost probe (§4.1) tells a practitioner which regime their loop is in on their device. The regime that train-time iterated resolving structurally generates — many small same-topology PBS subgames per iteration — is the launch-bound one.

**The end-to-end win survives Amdahl.** Microbenchmark speedups routinely evaporate inside real training loops. Ours does not, because the inner re-solve is 98.4% of the baseline loop at this operating point. On a tabula-rasa self-play resolving loop on Goofspiel-4 (3,608 information sets; 5 matched seeds; one configuration fixed in advance of the runs; the baseline arm is the *same* GPU SoA CFR+ kernel applied one tree at a time — i.e., the fair within-tree GPU-CFR baseline, not a Python strawman), the fused inner re-solve is  $11.79\times$  faster — against the  $11.7\times$  predicted ex ante by the §4.1 sweep — and *total* end-to-end wall-clock improves  $9.99\times$  (median of per-seed ratios; fused median 76.8 s vs sequential 767.2 s; convention in §3.4). The measured inner-resolve fraction then gives an Amdahl ceiling [3] of  $1/((1 - f) + f/S_{\text{inner}}) = 10.09\times$  — where  $f$  is the fraction of the sequential (baseline) arm’s total wall-clock spent in the inner re-solve and  $S_{\text{inner}}$  the fused-vs-sequential inner-resolve wall-clock ratio, each the median over matched seeds; the ceiling reading uses that the non-inner phases perform identical work in both arms (§3.4), so a seed’s total-wall ratio can exceed its own implied ceiling only through noise-scale non-inner timing asymmetry (§4.4) — and the measured  $9.99\times$  sits at 99% of it — the microbenchmark predicted the inner ratio ex ante, and the Amdahl decomposition closes the total-wall accounting to within 1%. At the other end (Goofspiel-5, where one subgame already fills the device), the microbenchmark’s saturated-regime prediction of  $2.37\times$  is measured in the end-to-end loop at  $2.40\times$  (median of 3 matched seeds on a quiet host; an earlier contaminated pair is disclosed as the motivating diagnostic in §4.4), and the same closure holds there: total wall improves  $2.00\times$  against a  $2.01\times$  ceiling at the measured 86.1% inner fraction. The speedup buys *no change in solution quality*: at Goofspiel-4

both arms finish in the same exploitability band (fused final NashConv — exploitability, defined in §3.4 — median 0.0402 [10th/90th: 0.0238, 0.0533]; sequential median 0.0469), and on CPU, where floating-point accumulation is deterministic, the two arms are provably the same algorithm. For calibration of what the resolving apparatus itself buys per unit compute, a fair search-free baseline (Neural Fictitious Self-Play (NFSP) with its average policy, one published configuration, not tuned) remains near uniform (NashConv 1.3968/1.3966 vs. uniform 1.4167) after 600 s — the fused arm reaches its  $\sim 0.04$  band in roughly 1/8th of that wall-clock — a directional boundary, not a converged comparison [20]. (Current-policy gradient methods without averaging oscillate near uniform and would be a strawman comparison [48]; the strongest search-free line we are aware of, VRPO, is a cited boundary we deliberately do not reimplement.)

**Scope, stated up front.** Our claim is *equal-band-faster* — the same exploitability band in less wall-clock — never lower exploitability, and never strength against poker bots. The headline comparison is same-substrate: fused-vs-sequential on one GPU, isolating the fusion mechanism. Two scoping facts from our own artifacts temper the cross-substrate reading and are presented, not buried (§3.3, §4.2, §5): at Goofspiel-4 scale the eager-dispatch GPU-sequential baseline is itself  $\sim 4.3\times$  slower per belief than the same kernel on the host CPU, so the best-fused-vs-best-non-fused reading at G4 is  $\sim 2.7\times$ , provisional on matched full-configuration CPU timings (the CPU cells are single runs without warmup/repeat discipline; §3.3) — in these runs, in the launch-bound regime, fusion is what made the GPU the faster substrate at all; and no CUDA-Graphs/`torch.compile` launch-amortized sequential arm has been measured yet (a planned ablation; the per-tree iteration is plausibly graph-capturable after a modest rewrite, §3.3). The demonstrated training loop is a 2-level, single-cut instance (per-belief value targets, independent given the belief); deeper self-play *correctness* is certified separately in our pipeline, but multi-level fused-vs-sequential *timing* is deferred to future work. Results are on one game family (the Goofspiel ladder; the fused solver is parity-tested on Leduc and liar’s dice but not end-to-end timed there) and one device (an RTX 3070 Ti, 8 GB, FP64 at 1:64 of FP32 throughput); the regime model is hardware-independent in form, and its parameters — including the regime boundary itself — are device-specific (§5). Exploitability at scale is reported as bands with seed error bars, not curves — the speedup claim is time-based and robust to that noise, since both arms share each seed. Section 5 expands each of these points.

## Contributions

- **Identification of the population structure (an unclaimed lever in game solving).** Train-time iterated resolving generates, at every iteration, a population of  $K$  same-topology depth-limited PBS subgames differing only in entry beliefs. Batching many small same-shape problems is standard numerical-systems practice (§2, systems prior art); claiming it for *populations of extensive-form subgame solves inside a resolving loop* is, to our knowledge (§2), unclaimed in published work — a lever orthogonal in form to the  $N_{\text{solves}}$ , iterations-per-solve, sample-variance, and subgame-size levers (composition untested).
- **A parity-gated fused solver.** A level-grouped, flat structure-of-arrays CFR+ that compiles the shared topology once and runs each iteration of *all* subgames as one sequence of batched tensor ops, recovering classical range-vector semantics [22, 23] across heterogeneous decision/chance/variable-action structure and across public keys. Equilibria match per-tree GPU-CFR exactly in the gated sense:  $0\text{--}7 \times 10^{-6}$  (float64), a CPU exact-parity gate ( $< 10^{-9}$ ), and CPU training trajectories identical to all recorded digits — at solve level the speedup is free (gated parity); through CUDA training the arms are band-equivalent, with per-seed divergence fully attributed to compounding atomics (§3.4).

- **A predictive two-regime cost model with an operationalized regime probe.** speedup  $\approx \min(N, 1/\varphi)$ , with  $\varphi$  the subgame fill fraction (the min form’s second argument sets the regime boundary, not the plateau height; §4.1): near-linear in the launch-bound many-small-subgames regime (fused time  $\sim$ constant, 197.6 $\rightarrow$ 202.4 ms, as  $N$ : 1 $\rightarrow$ 12) and saturating at  $\sim 2.4\times$  when one subgame fills the device. A one-measurement marginal-cost probe (fused  $N=1$  vs small  $N>1$ ; the boundary rule  $N \geq 1/m$ ; §4.1) classifies the regime from the released data, and the microbenchmark prediction transfers to the training loop at both ends: launch-bound G4, 11.7 $\times$  predicted  $\rightarrow$  11.79 $\times$  measured ( $n=5$ ); saturated G5, 2.37 $\times$  predicted  $\rightarrow$  2.40 $\times$  measured ( $n=3$ , quiet host; §4.4).
- **An Amdahl-surviving end-to-end training win.** In a real tabula-rasa self-play resolving loop (Goofspiel-4, 5 matched seeds, one configuration), the fused inner re-solve is 11.79 $\times$  faster and total wall-clock improves 9.99 $\times$  (fused median 76.8 s vs sequential 767.2 s), with the ex-post Amdahl decomposition from the measured 98.4% inner fraction closing the accounting to within 1% (ceiling 10.09 $\times$ ); final bands are equivalent (fused median 0.0402 [10th/90th: 0.0238, 0.0533] vs. sequential 0.0469).
- **A reusable evaluation methodology for batched GPU game solving.** Matched-seed paired arms sharing one kernel, CPU exactness gates, and the compounding-atomics finding: CUDA `index_add_` atomics compound through training into per-seed final-exploitability divergences (G4 median 0.0144, max 0.0389) far above any per-seed tolerance, so naive per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail — gate algorithmic equivalence on CPU, use GPU runs for timing and band-level comparison only (§3.4).

## 2 Related Work

**An organizing taxonomy.** The wall-clock cost of train-time iterated resolving factors as in Eq. (1): (number of solves)  $\times$  (iterations per solve)  $\times$  (wall-clock per CFR iteration of a solve), with sampling variance per traversal a further axis when traversals are stochastic. Prior work in game solving attacks the first two factors and the sampling axis; to our knowledge no published work claims the **cost-per-solve** factor (wall-clock per CFR iteration of a solve) for *populations* of subgames — the lever this paper isolates.

- **Number of solves ( $N_{\text{solves}}$ ).** TurboReBeL [34] is the nearest cost-efficiency cousin of this paper: it reports matching ReBeL’s exploitability at a small fraction of its training cost by amortizing target generation across CFR iterations — its primary lever is **single-sample multi-iteration generation** (one sampling pass yields data for all  $T$  iterations), with suit/chip isomorphism augmentation as a secondary, poker-specific lever. It reduces the number of solve/sampling passes per training datum; each solve is still executed one subgame at a time. The levers compose: theirs is per-datum solve count, ours is per-solve cost.
- **Iterations per solve.** DCFR [9] and PCFR+ [18] accelerate the convergence of a single solve by discounting and prediction. Meta-learned predictive regret minimization — originated in [50] and extended to self-play in [51] — trains over a *distribution* of subgames but deploys on each subgame *sequentially*: it reduces iterations per solve, not the cost of an iteration, and stacks with our lever. Deep-Predictive DCFR [56] — building on Deep CFR [10], the canonical neural CFR — and AlphaEvolve-discovered CFR variants [35] extend this line.



- **Samples per traversal.** MCCFR [30] and its variance-reduced descendants VR-MCCFR [46], ESCHER [38], and DREAM [49] reduce the variance (hence the count) of sampled traversals; the per-iteration arithmetic of each solve is untouched.
- **Subgame size.** LAMIR [28] learns abstractions that limit each resolved subgame to a manageable size, making principled look-ahead tractable in games where it previously could not scale. Smaller subgames *deepen* the launch-bound regime in which fusion pays most (regime model, §4.1).
- **Cost per solve (this work).** Fusing the population of same-topology depth-limited PBS subgames generated by one resolving sweep into shared batched kernel launches, with parity to per-tree CFR+ gated as in §3.4.

**GPU-accelerated CFR.** Kim [24] vectorizes CFR *within* a single game tree; our sequential baseline arm has exactly this shape (the same level-grouped SoA kernel, applied one subgame at a time). Kim & Sandholm [25] reframe CFR as a series of linear-algebra operations, and the authors’ NoRegret library (v0.0.0.dev15, released 2026-06-04) makes this an actively developed, still per-tree line; a GPU-CFR implementation for the card game Pasur [4] and the open-source gpucfr code (CTU Prague) [44] are further per-tree/per-game data points. As of 2026-06-09 we find no published work claiming population-level fusion on this line (a narrow, time-stamped statement). We note plainly that in the linear-algebra formulation of [25], a *mechanical* multi-tree batch dimension is an anticipatable engineering extension — a block-diagonal arrangement NoRegret could ship in a point release. What this paper claims is therefore the conjunction, not the batch dimension: the identification that *iterated resolving structurally generates* the population, the parity-gated loop integration with shared range-vector state, and the predictive regime model with its end-to-end validation. Those survive even if per-tree libraries add batching.

**Continual resolving and decision-time search.** DeepStack [41] introduced continual resolving with neural counterfactual-value leaves; what it batches are *net evaluations* at subgame leaves, not the solves themselves. Naive depth-limited solving is unsound [8]; safe-resolving gadgets [13, 40, 41] and safe-and-nested subgame solving [6] restore soundness, with Libratus [7] the canonical agent instance (and Pluribus [11] its multiplayer continuation, outside our two-player zero-sum scope). ReBeL [12] moved resolving into the training loop on public belief states — the loop whose inner solve we fuse — and Student of Games [47] unified search-based agents across game classes. Within this line, TurboReBeL and LAMIR attack  $N_{\text{solves}}$  and subgame size respectively (taxonomy above), and CDBR [39] resolves against an opponent model rather than to equilibrium. Kubicek, Lisy & Sandholm [29] show that resolving-gadget equilibria are non-unique and refinement-sensitive; accordingly, our exploitability-band claims use the on-policy re-solve continuation, with gadget numbers reported alongside (§4).

**Batched solving and adjacent claims in game solving.** Five nearby claims are distinct from ours. (i) Marris et al. [37] batch equilibrium solving, but with a *learned, amortized* solver over *normal-form payoff matrices*; we run parity-gated CFR+ over *extensive-form* depth-limited PBS subgames — variable depth, chance nodes, per-subgame regret and range state — inside a training loop. (ii) DeepStack’s batching is net-eval batching, not solve batching. (iii) The distribution-of-subgames in [51] is a meta-training set for a predictor that is then applied sequentially per subgame. (iv) US Patent 11,204,803 [33] decomposes the game into independently solvable subtasks — task-level parallelism, not fusion of many solves into shared kernel launches on one device. (v) The within-public-state range dimension — solving for all private hands simultaneously — is classical [22, 23] and is present in *both* of our timing arms; our claim is the across-key axis (§3.2).

**Systems prior art: batching small workloads.** The economics we exploit — many small same-shape problems under-utilize a GPU; fuse them into shared launches — is the founding insight of the batched-BLAS line [1, 2], and our  $\min(\cdot, \cdot)$  regime model is roofline-style throughput arithmetic [55]. CUDA Graphs [42] amortize launch overhead for a fixed kernel sequence without fusing work (the rival lever we discuss in §3.3); FlashAttention [16] set the mechanism + cost-model + end-to-end-win template we follow, with hardware-derived model parameters as the standard to meet. In the games-adjacent systems literature, mctx [15] batches MCTS search populations in the perfect-information family; Pgx [27], EnvPool [54], and Isaac Gym [36] vectorize game/environment *simulation*, not iterative coupled-state solving. None of these carries a shared cross-instance regret table through a clamped fixed-point solve under an exactness gate — that conjunction, in the extensive-form resolving loop, is what is new here; the batching pattern itself is not, and we do not claim it.

**Search-free self-play.** NFSP [20] and regret/Q-based policy gradients (RPG/QPG) [48] learn without decision-time search; our directional search-free boundary uses NFSP’s *average* policy, since evaluating the RPG/QPG *current* policy early in training is near-uniform and would be a strawman (§4.3). VRPO [17] reports a positive win rate against Slumbot in heads-up no-limit Texas hold’em (HUNL) — a result its authors describe as not a decisive win — without online search or subgame solving, and is the strongest search-free line we are aware of; we cite rather than reimplement it, because the load-bearing baseline for a cost-per-solve claim is the in-experiment sequential arm sharing the same kernel, the same seeds, and gated parity.

## 3 Method and Experimental Setup

### 3.1 Preliminaries: depth-limited PBS subgames and where the population arises

We consider two-player zero-sum imperfect-information games in the standard extensive form. A *public belief state* (PBS) is a pair  $\beta = (s_{\text{pub}}, (r_1, r_2))$ , where  $s_{\text{pub}}$  is a public state and  $r_i$  is player  $i$ ’s reach-probability vector over their private states consistent with  $s_{\text{pub}}$ . *Depth-limited solving* truncates the game at a set of cut points and replaces each truncated continuation with a leaf value function over the PBS at the cut; with arbitrary leaf values this is unsound [8], and the sound instantiations — DeepStack’s continual re-solving [41], ReBeL [12], Student of Games [47] — train a value network  $\hat{V}(\beta)$  and re-solve subgames at play time and, critically for this paper, at *train* time. Value-function error propagates to exploitability with known bounds [26].

Our subgame-construction layer (`DepthLimitedGame`) enumerates an OpenSpiel game [32] once into a tagged tree of terminal, chance, decision, and **cut** nodes. The cut predicate is the one game-specific hook; each cut node records the public key  $k$ , each player’s private index  $(i_0, i_1)$  at that key, and retains the continuation subtree. Leaf values use a fixed *normalized* PBS convention,

$$v_i(h_i \mid \beta) = \frac{\sum_{h_{-i}} r_{-i}(h_{-i}) \text{EV}_i(h_i, h_{-i})}{\sum_{h_{-i}} r_{-i}(h_{-i})},$$

where  $h_i$  ranges over player  $i$ ’s private states consistent with  $s_{\text{pub}}$ ,  $r_{-i}$  is the opponent’s entry reach vector of  $\beta$ , and  $\text{EV}_i(h_i, h_{-i})$  is player  $i$ ’s expected continuation payoff at the cut instance identified by the private pair  $(h_i, h_{-i})$  — under the re-solved subgame’s average strategy when generating targets, and under the exact full-game continuation in the round-trip gate below. The convention is validated by a round-trip gate: valuing the trunk through this leaf interface reproduces exact full-game above-cut values to  $\sim 10^{-15}$ , and breaks (large error) under the un-normalized convention. When the opponent entry reach vanishes ( $\sum_{h_{-i}} r_{-i}(h_{-i}) = 0$  — possible only for off-path trunk

queries, never for the strictly positive Dirichlet-sampled training beliefs of §3.4), the normalized value is defined as 0; the convention is inert, since any use of such a leaf value carries the same vanishing opponent-reach segment as a factor in its counterfactual weight.

**The training loop.** The tabula-rasa loop we accelerate is iterated continual resolving with *per-belief equilibrium targets*: in each round, (i) sample a belief  $\beta$  at every cut public state; (ii) **re-solve every below-cut subgame to equilibrium at the sampled belief** with CFR+ and read off the normalized continuation values  $V^*(\beta)$ ; (iii) regress the PBS value net on  $(\beta, V^*(\beta))$  rows; (iv) periodically re-solve the trunk against the current net leaf and measure exploitability of the assembled policy. Step (ii) is the *inner re-solve* and is where the cost lives.

**Where the population arises —  $K$  and  $B$ .** The inner re-solve is never one subgame. A single belief sample requires solving *all*  $K$  cut public keys’ subgames at once — on the Goofspiel ladder  $K = 9/12/15$  (§3.4) — and each key’s solve carries its private-deal instances as a range dimension, for  $B = 27/64/125$  cut instances in total. The within-key range dimension is the classical public-state vectorization of CFR [22, 23] and is batched identically in both of our timing arms; **the fusion axis of this paper is  $K$**  — the across-key population — and  $K$  ceilings the cost model and the measured median speedups; per-seed values can exceed it slightly through measurement noise and asymmetric launch overhead (§4.1, §4.2). Target generation repeats the  $K$ -population solve for every sampled belief in every round. This is the canonical workload of the method family: DeepStack’s value networks were trained on the order of  $10^7$  randomly sampled subgames, each re-solved to equilibrium [41]. In terms of the cost identity of Eq. (1), prior work attacks  $N_{\text{solves}}$ ,  $T$ , the sampling variance, or the subgame size; this paper targets the third factor, **cost-per-solve**, by fusing the population into one batched solve. The levers are multiplicative and orthogonal in form; composition with the others is untested (§5).

### 3.2 Mechanism: level-grouped compilation and flat-SoA CFR+ over the population

**Shared topology, compiled once.** All cut-node continuation subtrees within a game share a single topology (the cut sits at a public chance boundary; instances differ only in private deals, hence in entry reaches, payoffs, and info set identities). The topology compiler exploits this with one recursive descent over the *lockstep population*: it emits one record per topology node, whose per-instance data are stored as length- $B$  arrays — terminal payoffs  $[B, 2]$ , chance branch probabilities  $[B, n_{\text{branch}}]$ , and per-instance global info set ids  $[B]$  at decision nodes. Records are then grouped **by level and shape**: forward groups collect all level- $L$  children by parent kind (decider 0, decider 1, chance); backward groups collect all level- $L$  non-terminals by (player,  $n_{\text{actions}}$ ) or (chance,  $n_{\text{branch}}$ ). Variable depth, heterogeneous branching factors, and interleaved chance nodes are absorbed at compile time into these groups plus padded legal-action masks over the union of below-cut info sets.

**Flat-SoA CFR+ iteration.** *Notation:*  $B$  cut instances and  $K$  public keys as in §3.1;  $n_I$  below-cut info sets in the population union with at most  $A_{\text{max}}$  actions each;  $R, S \in \mathbb{R}^{n_I \times A_{\text{max}}}$  the global regret and strategy-sum tables;  $\sigma$  the current strategy from regret matching;  $r^{\text{own}}, r_{-i}, r^c$  own, opponent, and chance reaches;  $G$  denotes the node count of whichever level-shape group an op ranges over. Each CFR+ iteration is then a fixed sequence of batched tensor ops with *no per-iteration Python tree recursion*:

1. *Regret matching* [19]: one clamped-normalize over the global regret table  $R$  (float64),  $\sigma(I, a) = [R(I, a)]_+ / \sum_{a'} [R(I, a')]_+$  over legal actions, uniform over legal actions where the denominator vanishes (illegal slots are mask-padded out of every update).



2. *Forward reach pass*: for each forward group, child reaches are one gather + elementwise multiply —  $r_{\text{child}}^{\text{own}} = r_{\text{parent}}^{\text{own}} \odot \sigma[\text{id}, \text{slot}]$  for decision parents, where  $\sigma[\text{id}, \text{slot}]$  is the parent info set’s current-strategy entry for the action leading to the child, and  $r_{\text{child}}^c = r_{\text{parent}}^c \odot p$  for chance parents, where  $p$  is the branch probability — over  $[G, B]$  blocks.
3. *Backward value pass*: for each backward group, child EVs are gathered as  $[G, A, B, 2]$  tensors, where  $A$  is the group’s per-info set action (or chance-branch) count, and contracted against  $\sigma$  (or chance probabilities); counterfactual action values  $r_{-i} r^c$  EV (opponent reach times chance reach — the counterfactual reach) are scattered into the global accumulator with `index_add_` over the per-instance info set ids.
4. *CFR+ update*: alternating-player regret update with clamp at zero, own-reach-weighted strategy sums, linear averaging (on the interaction of alternating updates with averaging in CFR+, see [14]).

Per-iteration kernel-launch and op count is  $O(\# \text{level-shape groups})$  — fixed by the topology and *independent of the population size*, which appears only as a tensor dimension (the arithmetic work per op scales with it, which is precisely what fills the device). (We use “GEMM-shaped” loosely: the per-iteration ops are gathers, scatters, and elementwise products plus small dense contractions, executed as batched tensor kernels.) This is the entire mechanism: the population axis is moved from a host-side loop (one kernel-launch sequence per subgame) into the innermost tensor dimension of every op.

**Reuse across resolves.** Between solves only the entry reaches change: the two  $[B]$  entry-reach vectors are rewritten and the compiled topology is reused for the whole training run. Compilation is one-time infrastructure and is excluded from all solve timings symmetrically in both arms — conservatively for the fused claim, since the sequential arm requires  $K$  per-key compiles plus the shared one (quantified compile walls remain a planned instrumentation artifact, alongside the §3.3 ablations). After the final iteration, a value-only backward pass under the average strategy returns the per-instance continuation EV  $\text{cont} \in \mathbb{R}^{B \times 2}$ , from which normalized per-private leaf values are reconstructed in  $O(B)$  — so target generation and trunk leaf queries also avoid any Python subtree walk.

**Parity, and what is and is not nontrivial about it.** We require the fused solve to return the *same equilibrium* as the per-tree solve — exact in the gated sense defined below. Three clarifications, stated plainly because they delimit the novelty claim. First, *the within-key coupling is classical*: within one public key, multiple cut instances write reach-weighted counterfactual values into the same info set rows — this is range-vector CFR [22, 23], present identically in both arms, and not what fusion adds. *Across* keys, the below-cut info set rows are disjoint (post-cut info sets condition on the public outcome), so across-key fusion changes only floating-point accumulation order and op scheduling — which is exactly why parity is mathematically expected (Proposition 1); the observed parity is 0 at Goofspiel-3 (G3) and G4, and  $4 \times 10^{-6}$  at G5.

**Proposition 1** (across-key parity of fused CFR+). *Fix a compiled population with public keys  $k \in \{1, \dots, K\}$ , cut instances partitioned as  $\mathcal{B} = \biguplus_k \mathcal{B}_k$ , and below-cut info set (regret-row) sets  $\mathcal{I}_k$  that are pairwise disjoint — as holds whenever each below-cut info set determines its public key, i.e., the public outcome at the cut is observed by both players. Run (a) the fused solver on all  $K$  keys and (b) the same CFR+ kernel on each key alone, with identical entry reaches, iteration budget, alternating-player updates, clamp at zero, linear averaging, and the same masked regret-matching rule (uniform over legal actions when the clamped row sum vanishes — implemented as a fixed threshold (positive-regret row sum  $> 10^{-12}$ , the same threshold guarding the final average-strategy*

normalization), identical in both arms). Then, in exact arithmetic, for every iteration  $t$  the fused iterates — regrets, current strategies, strategy sums, and all reach and counterfactual-value quantities — restricted to rows  $\mathcal{I}_k$  and instances  $\mathcal{B}_k$  equal the per-key iterates of key  $k$ ; in particular both arms return identical average strategies and leaf values.

*Proof sketch.* Induction on  $t$  over the four steps of the iteration. (i) Regret matching acts row-wise, so equal rows give equal current strategies on  $\mathcal{I}_k$ . (ii) The forward pass computes each instance’s reaches from its own parent’s reaches and the parent’s strategy (or chance) entries — purely per-instance data, and every instance lies in exactly one key. (iii) The backward pass scatter-adds per-instance counterfactual values into rows indexed by per-instance info-set ids; by disjointness of the  $\mathcal{I}_k$ , each row receives contributions only from instances of its own key, so the fused scatter restricted to  $\mathcal{I}_k$  sums exactly the multiset of addends of key  $k$ ’s solo solve. (iv) The CFR+ update (clamp, alternating-player gating, own-reach-weighted strategy sums, linear averaging) is again row-wise, and padded action slots are masked out of every update and stay zero, so the arms’ differing pad widths do not affect legal-slot values. The own-reach weight entering the strategy-sum update is written by a scatter-assignment over per-instance info-set ids rather than an add; under perfect recall every instance of an info-set carries the same own-reach value (an info-set determines its owner’s private state and own action sequence, hence, within one solve, the same entry reach and strategy product), so the assignment is well-defined, and by disjointness of the  $\mathcal{I}_k$  the fused write restricted to  $\mathcal{I}_k$  coincides with key  $k$ ’s. Hence the fused update is the direct sum over keys of the per-key updates. In floating point the two arms therefore sum the same per-row multisets of addends and can differ only through intra-row accumulation order and op scheduling — which the CPU parity gate bounds at  $< 10^{-9}$  and the CUDA runs realize at  $0\text{--}7 \times 10^{-6}$  (§3.4).  $\square$

Second, the *genuine* engineering nontriviality is elsewhere: recovering these semantics through a level-grouped compilation of irregular structure (variable-depth branches, mixed chance/decision levels, per-info-set action counts, mask padding); float64 discipline through hundreds of compounding clamped fixed-point iterations with linear averaging (a float32 ablation drifted by  $\sim 0.75$  L1 in average strategy at repeated imperfect-information info-sets); and integration into a live training loop with topology reuse and  $O(B)$  leaf reconstruction. Third, parity is a *gated property*, not a by-construction guarantee: CFR+ amplifies ordering and padding discrepancies rather than averaging them out, so the gates of §3.4 are part of the method.

### 3.3 Baseline: the same kernel, one tree at a time

The baseline arm (**sequential**) is the *same* flat-SoA CFR+ kernel applied to one subgame tree at a time, each with its own precompiled, reused topology — i.e., within-tree GPU-CFR in the sense of [24]: every individual subgame is solved on-GPU with full within-tree parallelism (including the within-key range dimension), but the population is traversed by a host loop with one launch sequence per tree. The two arms execute literally the same code path and differ only in batching granularity (guarded by an assertion in the run harness). We explicitly *reject* the Python reference tree-walk solver as a baseline: it is the correctness reference, but timing against it would inflate the speedup with interpreter overhead and make the comparison a strawman. Any reported fused-vs-sequential speedup is thus attributable to population fusion alone, not to GPU-vs-CPU or compiled-vs-interpreted effects.

**What the baseline is not.** Two stronger non-fused baselines exist and bound the claim; we present both rather than leaving them to a skeptical reader.

(i) *The host CPU.* The four cells of the per-belief inner-resolve  $2 \times 2$  at Goofspiel-4 have two sources. The CPU cells come from our CPU determinism-gate artifacts (§3.4), which contain timings

of the same two arms on the host CPU at a reduced budget (4 rounds  $\times$  10 beliefs; one run per cell; per-belief values are the run’s inner-resolve total divided by its 40 resolved beliefs — per-belief inner cost is budget-invariant, since each belief is an independent re-solve of the same population at the same 300 CFR+ iterations, the default setting; the gate artifacts do not record the iteration count, so the §7 reproduction line pins it explicitly). The GPU cells come from the seed-0 full-configuration end-to-end runs of §4.2 (inner-resolve totals divided by their 320 resolved beliefs; seed-0 values, not the §3.4 median convention). The  $2 \times 2$ : CPU-sequential 0.547 s, CPU-fused 0.42 s, GPU-sequential 2.353 s, GPU-fused 0.201 s. That is: **at G4 scale the eager-dispatch GPU-sequential baseline is  $\sim 4.3\times$  slower per belief than the same kernel on the host CPU**, fusion on CPU is worth  $1.30\times$ , and the best-fused-vs-best-non-fused comparison (GPU-fused vs CPU-sequential) is  $\sim 2.7\times$  — the practitioner-facing cross-substrate number at G4. Caveats, both directions: the CPU cells come from determinism-gate runs, not a dedicated timing protocol (no warmup/repeat discipline), and the two protocols differ, so the  $\sim 2.7\times$  is provisional on matched budgets; matched full-configuration CPU timings at G4 and G5 are planned. The reading we draw is the regime model’s own: in the launch-bound regime — tiny float64 subgames — eager per-tree GPU dispatch lost even to the host CPU, and *in these runs fusion is what made the GPU the faster substrate at this scale*. At G5 no CPU timings exist; with 236,450 infosets the arithmetic-dominated comparison is expected to invert, but that is untested and we do not claim it.

(ii) *A launch-amortized sequential arm (CUDA Graphs / `torch.compile`)*. The launch overhead that fusion amortizes is also the target of CUDA-Graphs capture and `torch.compile(mode="reduce-overhead")`. We have read our own kernel against capturability rather than asserting either way. Per compiled topology, every tensor shape in the solver’s iteration body is static across iterations and across re-solves (the level-shape groups, the  $[n_I, A_{\max}]$  tables, the  $[n_{\text{nodes}}, B]$  reach tensors; re-solves rewrite entry-reach values only), and the loop contains no host-side convergence checks or synchronizing `.item()` reads — so there is **no structural barrier to capture**. The real obstacles are engineering-level: (a) two Python scalars depend on the iteration index — the alternating-player update parity ( $t \bmod 2$ ) and the linear-averaging weight  $w = t+1$  — which a capture would bake in (standard remedy: a pair of parity-alternating graphs plus a device-resident weight); (b) the regret/strategy-sum updates are out-of-place tensor rebindings with fresh per-iteration intermediates, which capture/replay requires rewriting into static in-place buffers (or delegating to the compile-mode graph pool). We therefore do **not** claim inapplicability: a graph-captured or compiled per-tree sequential arm is plausibly buildable, and the measured ablation — how much of the launch-bound gap graphs recover, versus how much is short-kernel occupancy that only fusion’s larger per-kernel work addresses — is planned alongside the full-protocol ( $n \geq 5$ ) G5 band run. Our prediction, stated for the record: capture can recover the launch-gap component but not the short-kernel occupancy component, so it should close part — not all — of the launch-bound gap. Until that ablation is complete, the launch-bound speedups of §4 are measured against an eager-dispatch single-stream sequential arm, and we say so wherever they are claimed.

### 3.4 Experimental setup

**Games.** The Goofspiel ladder [43] (OpenSpiel, simultaneous-move game loaded through the turn-based wrapper; imperfect information, random prize order) with 3, 4, and 5 cards (G3/G4/G5). The depth-limit cut is the prize-reveal chance boundary after the first resolved round (a single cut level; see the scope paragraph in §5). The ladder spans:

At G5 the compiled topology has 26,773 nodes over 10 level groups and the fused solve runs at  $\approx 6.2$  ms per CFR+ iteration in-loop, including leaf reconstruction — indistinguishable from the pure-solve microbenchmark figure ( $\approx 6.2$  ms/iter). The implementation is game-agnostic in interface

| game | public keys $K$ (fusion axis) | cut instances $B$ | infosets |
|------|-------------------------------|-------------------|----------|
| G3   | 9                             | 27                | 90       |
| G4   | 12                            | 64                | 3,608    |
| G5   | 15                            | 125               | 236,450  |

Table 1: The Goofspiel ladder: population dimensions per game.

(the cut predicate and public-key function are the only game hooks; correctness is parity-tested on Leduc poker and liar’s-dice cut structures), but performance generality is demonstrated only on Goofspiel: all end-to-end timing claims in this paper are on this ladder.

**Metric.** Exploitability is *exact* NashConv [31],  $\sum_i (\max_{\sigma'_i} u_i(\sigma'_i, \sigma_{-i}) - u_i(\sigma))$ , where  $u_i$  is player  $i$ ’s expected utility under the assembled profile  $\sigma$  ( $\sigma$  here denotes the assembled full profile, distinct from the within-solve current strategy of §3.2), computed by OpenSpiel’s exact best response on the assembled full policy: the trunk average strategy spliced onto the below-cut continuation re-solved at the on-policy belief. We additionally report the DeepStack-style safe-resolving gadget continuation [13, 41]; since gadget equilibria are refinement-sensitive [29], band claims use the on-policy re-solve continuation, with gadget numbers alongside. No sampled or approximate exploitability is used anywhere.

**Paired arms, one configuration.** The end-to-end experiment runs the §3.1 loop twice per seed — once with the fused inner solver, once with the sequential inner solver — under **one** configuration fixed in advance of the runs, with no sweeps (verifiable from the released commit history): 8 target rounds  $\times$  40 sampled beliefs per round; 300 CFR+ iterations per inner re-solve; beliefs drawn per key as symmetric Dir( $\alpha$ ) over each player’s private states, with  $\alpha$  resampled uniformly at random from  $\{0.3, 1.0, 3.0\}$  independently per player, per key, per sampled belief; trunk CFR+ 200 iterations; PBS value net = MLP (hidden 128), 400 epochs per round, buffer cap 8,000; exact measurement every 2 rounds and at the end (on-policy continuation re-solved with 600 iterations; gadget 1,500). Arms are *matched-seed*: both arms of a pair share the seed, hence the identical belief sequence and (up to CUDA atomics, below) identical targets and trajectories. G4 uses 5 paired seeds. **Aggregation convention** (used throughout): every fused-vs-sequential end-to-end speedup in §4 is the median of per-seed ratios; the §4.1 microbenchmark speedups are ratios of median-of-7-repeat timings (no seed axis); and the secondary time-to-band speedups (§4.2, §4.4) are ratios of the per-arm median reach times, since per-seed time-to-band ratios are quantized by the every-2-rounds measurement cadence. Where  $n \leq 2$  we report the per-seed values themselves and do not dress the midpoint as a robust statistic. Ranges such as “5.5–11.4 $\times$ ” round the underlying values (5.48–11.36) to one decimal; all underlying values are in the artifacts. **Band equivalence:** two arms finish in the same exploitability band when each arm’s median final NashConv lies within the other arm’s per-seed 10th/90th interval. The criterion is deliberately descriptive and weak: at  $n=5$  each arm’s 10th/90th interval spans nearly its entire sample, so median gaps substantially larger than the one observed would also have passed — it certifies the absence of gross divergence, not statistical equivalence (§4.2). At G5 — the saturated regime of §4.1 — the same paired-arm design is run with a shortened budget (4 target rounds  $\times$  40 beliefs; exact measurement at the final round only; the safe-resolving gadget evaluation is skipped); all other settings are unchanged, and the results are reported in §4.4. The headline G5 numbers come from a quiet-host re-run of this design at 3 paired seeds (seeds 0–2; machine otherwise idle); the original 2-seed pair, whose sequential seed-1 arm was host-load-contaminated, is superseded and retained as the motivating diagnostic (§4.4). The shortening was a wall-clock budget decision taken before that contamination was discovered, not a protocol response to any result; a full-protocol (8-round) G5 paired run at

$n \geq 5$  with a pre-registered band margin — needed for a CUDA band-equivalence test, not for the timing claim — is planned. The 5-seed G5 exploitability band of §4.4 uses the full 8-round configuration above (seeds 0–4, fused path). **Total wall-clock (total\_wall)**: the wall-clock of the training loop — inner re-solve + net train + measurement phases, identically bounded in both arms; it excludes game enumeration, one-time topology compilation, and the post-loop gadget pass (all symmetric or conservative for the fused claim; §3.2).

**Timing discipline.** GPU sections are timed with `torch.cuda.Event` pairs bracketed by `torch.cuda.synchronize()`, with wall-clock recorded alongside. One sharp caveat we state as a finding (encountered in our G5 runs, §4.4): **GPU-event spans that bracket a host-driven launch loop are host-load-sensitive** — the event pair measures stream-timeline elapsed time *including* gaps where the device waits for host submission, so under host CPU contention a launch-heavy (sequential) arm’s event spans inflate while a fused arm, issuing far fewer launch sequences, stays fed. (Consistent with this, the recorded GPU-event totals equal the wall-clock totals to the millisecond in these runs — the events bracket wall-equivalent regions.) The cost-per-solve microbenchmark (Figure 1) uses 3 discarded warmup runs and the median of 7 repeats per point, on precompiled topologies (pure solve time). In the training loop, per-phase timers separate inner-resolve / net-train / measurement; the timed inner-resolve region covers exactly the per-belief population solve plus leaf reconstruction, while host-side row assembly (identical work in both arms) and one-time compilation are excluded. Measurement rounds run through the fused path in *both* arms — the evaluation protocol is fixed and identical — and their cost is tracked separately from the training-loop wall-clock comparison.

**Exactness gates, and why equivalence is gated on CPU.** Two hard gates certify that the two timing arms are the same algorithm. (1) A CPU exact-parity unit test (`test_fused_vs_sequential_soa_parity`) fused-over-all-keys vs sequential-per-key average strategies *and* leaf values agree to  $< 10^{-9}$ . (2) A CPU determinism run of the training loop in both modes — the full loop structure at a reduced budget (4 rounds  $\times$  10 beliefs; disclosed here and in §7): the trajectories (validation MAE per round, measured exploitability) are **identical to all recorded digits** (values recorded to 5 decimal places; e.g., per-round exploitability 0.14935/0.14817, identical in both arms). On CUDA, however, `index_add_` uses atomics, so floating-point accumulation order is non-deterministic; across 8 rounds  $\times$  40 beliefs  $\times$  300 iterations these perturbations *compound through training*, producing per-seed fused-vs-sequential final-NashConv differences of median 0.0144 (max 0.0389) at G4. The primary evidence that this divergence is unbiased is the CPU determinism gate (trajectories identical to all recorded digits — the two arms are the same algorithm) together with the symmetry of atomic scheduling noise; the G4 seed signs (fused higher on 2/5, lower on 3/5) are *consistent* with no bias but unpowered as a conclusion — **no detectable bias ( $n=5$ ; a sign test at this  $n$  cannot exclude moderate bias)**, and pooling the quiet-host G5 pairs gives fused-higher on 4/8 (§4.4). We therefore gate algorithmic equivalence on CPU, where it is exact, and use CUDA runs for timing and *distributional* (band-level) comparison only. We flag this compounding-atomics effect as a finding in its own right: naive per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail.

**Search-free boundary protocol.** As a per-compute sanity boundary we run NFSP [20] — the canonical search-free near-Nash learner — on the same game object, scored by the same exact NashConv, under one published OpenSpiel configuration (not swept, not Goofspiel-tuned), evaluated in average-policy mode, with a pre-registered criterion, recorded in the artifact, under which the baseline would be replaced. NFSP executes on the host CPU (the OpenSpiel PyTorch implementation’s default device, which our run does not override), while both resolving arms use the GPU — the §4.3 wall-clock comparison therefore crosses platforms, one more reason it is directional only. We choose NFSP over a tuned PPO-with-averaging baseline — which Rudolph et al. [45]



| game          | fused (ms) | sequential (ms) | speedup               |
|---------------|------------|-----------------|-----------------------|
| G3 ( $K=9$ )  | 119.2      | 1,045.6         | <b>8.77</b> $\times$  |
| G4 ( $K=12$ ) | 203.1      | 2,374.3         | <b>11.69</b> $\times$ |
| G5 ( $K=15$ ) | 1,866.7    | 4,427.4         | <b>2.37</b> $\times$  |

Table 2: Cost-per-solve microbenchmark: fused population solve vs. sequential per-tree GPU-CFR (GPU-event medians of 7 repeats, 3 warmups discarded, 300 CFR+ iterations).

suggest can be stronger per unit compute — precisely because NFSP has a published untuned configuration and an explicit averaged policy; the boundary is directional, and tuning any search-free baseline ourselves would invite an unfair-tuning critique in either direction. A methodological note we believe generalizes: regret/quantal policy gradient (RPG/QPG [48]) keep no averaged policy, so their *current-policy* NashConv oscillates near uniform (G4:  $\approx 1.4334$  vs uniform 1.4167; recorded in the supplementary measurement notes, §7; a standalone artifact is planned); reporting that as “the search-free boundary” would artificially inflate our advantage, and we reject it. The strongest search-free line we are aware of (VRPO [17]) is a cited boundary, not reimplemented — the load-bearing baseline in every experiment remains the sequential arm itself: the same kernel, with gated parity.

**Hardware.** All timings: one consumer GPU (NVIDIA RTX 3070 Ti, 8 GB; FP64 throughput 1:64 of FP32), PyTorch float64 tensor ops; exactness gates run the identical code on CPU. The mechanism and the regime model of §4 are stated hardware-independently; the model’s *parameters* — launch overhead, device capacity, and hence the regime boundary — are device-specific (§5 discusses the FP64 asymmetry’s direction), and a multi-device replication is deferred to future work.

## 4 Results

We present four results, in the order of the argument: the cost-per-solve microbenchmark and the two-regime model it induces (§4.1); the end-to-end training win at Goofspiel-4, where the model’s launch-bound prediction is tested against a real training loop (§4.2); a directional search-free calibration (§4.3); and the Goofspiel-5 scale results — the 5-seed exploitability band that the fused solver makes affordable, and the paired end-to-end point at the *saturated* end of the regime model (§4.4). Every speedup follows the §3.4 aggregation convention; every exploitability number is exact NashConv; **every fused-vs-sequential claim is equal-band-faster** — at G4 the band equivalence is tested on the CUDA runs themselves (§4.2), while at G5 the equivalence certificate is the CPU gates and single-solve parity (§4.4), the CUDA band test at G5 being pending the pre-registered full-protocol run (the one comparison of a different kind — the  $2.07\times$  target-construction result in §4.4 — is delimited there as such).

### 4.1 Cost-per-solve: microbenchmark and the two-regime model

We first measure the cost-per-solve lever in isolation: solve the full cut population of each Goofspiel-ladder game once with the fused solver (all  $K$  keys in one batched solve) and once with the *same* SoA kernel applied per key — one subgame tree per public key, i.e., the per-tree within-tree GPU-CFR baseline of §3.3. Timings are GPU-event medians of 7 repeats after 3 discarded warmups, 300 CFR+ iterations, on precompiled topologies ( $K$ ,  $B$ , infosets per game in the §3.4 ladder table).

Equilibrium parity is exact in the gated sense: maximum average-strategy difference between the fused and sequential solves is 0 at G3/G4 and  $4 \times 10^{-6}$  at G5 (float64; the range across all

parity-tested games, including Leduc and liar’s dice, is  $0\text{--}7 \times 10^{-6}$ ). The speedup costs nothing in solution quality.

The non-monotone speedup column ( $8.77 \rightarrow 11.69 \rightarrow 2.37$ ) is not noise — it is the regime structure, and the median speedups are ceilinged by  $K$ , the across-key fusion axis. A controlled sweep holding per-subgame size fixed and varying the number  $N$  of co-solved subgames (keys) — the *key sweep* — separates the two regimes. On G4, whose individual subgames under-fill the device, fused time is essentially flat — 197.6, 199.5, 200.1, 201.4, 202.4 ms at  $N = 1, 2, 4, 8, 12$  — while sequential time grows linearly, so the speedup tracks  $N$  nearly exactly:  $1.0/2.0/3.96/7.9/11.7\times$  (the key sweep is a separate run from the microbenchmark table row; their values agree within 0.4%). This is the *launch-bound* regime: the device is paying per-launch overhead, not arithmetic, and fusion amortizes the launches (against an eager-dispatch sequential arm; §3.3(ii)). On G5, whose deeper trees individually saturate the device, fused time grows with  $N$  ( $291 \rightarrow 1,870$  ms at  $N = 1 \rightarrow 15$ ) and the speedup saturates:  $1.0/1.59/2.08/2.24/2.39\times$  at  $N = 1, 2, 4, 8, 15$  — residual launch-overhead recovery on top of an already-busy GPU (the sweep endpoint  $2.39\times$  and the microbenchmark table row  $2.37\times$  are separate runs agreeing within 1%; the table row is the value used as the §4.4 prediction). The data are summarized by the two-regime model

$$\text{speedup}(N) \approx \min(N, 1/\varphi),$$

where  $\varphi \in (0, 1]$  is the *fill fraction* — the fraction of the device one subgame’s kernels occupy (we avoid CUDA’s narrower warps-per-SM sense of “occupancy”). The two branches have different statuses. The launch-bound branch is quantitative: while  $N\varphi \lesssim 1$ , fused time is flat in  $N$  (197.6–202.4 ms), so speedup tracks  $N$ . The second argument fixes only the regime boundary  $N^* \approx 1/\varphi$ , not the plateau height: once the device is arithmetic-bound, the residual gain is launch-overhead recovery rather than co-scheduling, so the plateau is an empirical parameter of the model, which we take from the microbenchmark itself ( $2.37\times$  at G5 on this device);  $\min(\cdot, \cdot)$  is the zero-overhead envelope — the speedup co-scheduling alone would deliver if kernel launches were free. The measured saturated speedup sits above this envelope’s second branch precisely because launch-overhead recovery persists once the device is arithmetic-bound, which is why the plateau is an empirical parameter of the model rather than  $1/\varphi$ . The min form is also monotone non-decreasing in  $N$  by construction: effects beyond launch amortization (e.g., memory traffic at very large populations) lie outside the envelope’s scope (§5). The model is additionally subject to memory capacity: the fused population’s state must fit on the device, so the attainable  $N$  is capped by a device-specific maximum population  $B_{\max}$  — every population in this paper fits on the 8 GB device (up to  $K=15$ ,  $B=125$ ), and peak-VRAM-per-arm instrumentation is a deferred reporting item (§5).

**An operationalized regime probe.** The regime — though not the exact plateau height — is classifiable from one cheap measurement: the **fused marginal-cost probe**. Solve the population fused at  $N=1$  and at a small  $N>1$  and compute the normalized marginal cost  $m = (\text{fused}(N) - \text{fused}(1))/((N-1) \cdot \text{fused}(1))$ . Under the affine extrapolation  $\text{fused}(N) \approx \text{fused}(1) \cdot (1 + (N-1)m)$ , the probe estimates the cost model’s own regime boundary:  $N^* \approx 1/m$  plays the role of  $1/\varphi$ , and the classification compares the population size  $N$  to be fused against  $1/m$ . A workload is launch-bound when  $N \ll 1/m$  — added subgames ride along nearly free and speedup tracks  $N$  — and saturated when  $N \gtrsim 1/m$  — fused time has grown by a multiple of the single-population solve and speedup plateaus at residual launch recovery. On the released data the probe’s classifications at the populations actually fused match the sweep-curve labels for all four sweep-covered workloads on this device (probe step = the smallest sweep point above  $N=1$ :  $N=2$  for G4 and G5,  $N=4$  for the HUNL classes); since the labels are read off the same sweep curves, this is an operational definition illustrated on released data, and the probe’s predictive content is carried by the microbenchmark-to-loop transfers of §4.2 and §4.4: G4  $m = 0.010$  (launch-bound; measured in-loop  $11.79\times$ , §4.2);

Population fusion: near-linear when launch-bound, saturating when GPU-bound

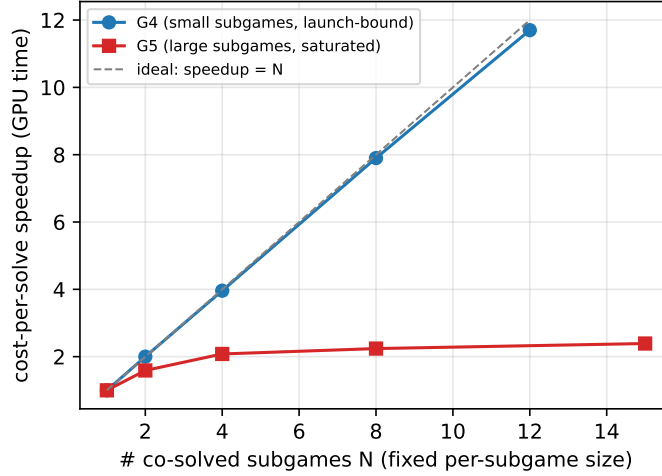


Figure 1: **The two-regime cost model in one picture.** Cost-per-solve speedup (GPU-event time, fused vs sequential per-tree GPU-CFR; the sequential arm is bracketed end-to-end around its per-tree host loop) as a function of the number  $N$  of co-solved subgames (public keys) at fixed per-subgame size. Goofspiel-4 (G4; small subgames, blue) is launch-bound: fused time is  $\sim$ constant (197.6  $\rightarrow$  202.4 ms for  $N$ : 1 $\rightarrow$ 12), so speedup tracks the ideal speedup =  $N$  line (dashed). Goofspiel-5 (G5; large subgames, red) saturates the device at  $N=1$ : speedup plateaus at  $\sim 2.4\times$ . Source: `costpersolve_bakeoff.json` (keysweeps; median of 7 repeats, 3 warmups discarded, 300 CFR+ iterations).

G5  $m = 0.277$  (saturated; plateau  $\sim 2.4\times$ , §4.4); HUNL census light river trees  $m = 0.062$  (launch-bound;  $10.86\times$  at 16 trees, §5); HUNL heavy river trees  $m = 0.278$  (saturated;  $3.35\text{--}3.74\times$ , §5). The light-river class sits essentially at the probe’s regime boundary at its measured 16-tree point ( $N \approx 1/m$ ), consistent with the recession at the larger co-solved population reported in §5. The model makes two predictions tested in the rest of this section: at G4 the in-loop inner-resolve speedup should be  $\sim 11.7\times$  (the  $N=12$  sweep point; §4.2), and at G5 it should be  $\sim 2.4\times$  (the saturated plateau, microbenchmark value  $2.37\times$ ; §4.4). Train-time iterated resolving structurally generates the launch-bound regime — many small same-topology PBS subgames per iteration — and the abstraction-learning line [28] pushes workloads further into it.

## 4.2 End-to-end training win, Goofspiel-4

Microbenchmark speedups routinely die inside real training loops; this one survives, because the loop is dominated by exactly the phase we fused. Running the full §3.1 loop under the single advance-fixed configuration of §3.4, with both arms sharing each of 5 seeds:

- **Inner-resolve speedup  $11.79\times$**  (median of per-seed ratios; per-seed range 11.68–12.23) — against the regime model’s launch-bound prediction of  $\sim 11.7\times$  from the §4.1 sweep, made before these runs.
- **Total wall-clock speedup  $9.99\times$**  (per-seed range 9.97–10.41): median total wall **fused 76.8 s vs sequential 767.2 s** per arm.
- **The Amdahl arithmetic closes ex post.** The inner re-solve is **98.44%** of the sequential

baseline loop (98.4% rounded elsewhere); given the measured inner speedup ( $S_{\text{inner}} = 11.79\times$ ; the ex-ante prediction was  $11.7\times$ ), the implied ceiling is  $1/((1-f) + f/S_{\text{inner}}) = 10.09\times$  (computed from the unrounded artifact value  $f = 0.9844$ ; this is the *median*-implied ceiling — each seed’s ratio is bounded by its own seed-level ceiling, which varies with that seed’s  $f$  and  $S_{\text{inner}}$ , and all five seeds satisfy theirs in the artifacts), and the measured  $9.99\times$  sits at 99% of it. To be precise about what is prediction and what is accounting: the *microbenchmark predicted the inner-resolve ratio ex ante* (the microbenchmark runs predate the loop runs; the ordering is verifiable from the released commit history, §7); the Amdahl decomposition uses  $f$  measured from the loop itself and is an accounting-consistency closure — the residual 1% reflects the non-inner phases together with the across-seed median aggregation.

- **Band equivalence, not improvement.** Fused final NashConv median **0.0402** [10th/90th: 0.0238, 0.0533]; sequential median **0.0469** [10th/90th: 0.0313, 0.0604, derived from the released per-seed artifacts]. The arms satisfy the §3.4 band-equivalence criterion — each arm’s median lies within the other arm’s per-seed 10th/90th interval — and we make no lower-exploitability claim. The criterion is a descriptive band check at  $n=5$ , not an equivalence test; TOST-style equivalence testing is deferred to the pre-registered full-protocol G5 run.
- **Secondary, threshold-sensitive view (time-to-band).** A time-to-band  $\tau$ -sweep ( $\tau \in [0.05, 0.12]$ ) — where time-to-band( $\tau$ ) is the cumulative inner-resolve-plus-net-train wall-clock (measurement phase excluded) to the first measurement round whose on-policy NashConv is  $\leq \tau$  — gives speedups from  **$5.5\times$  to  $11.4\times$**  depending on the threshold. At the pooled band ceiling  $\tau = 0.0598$  (the 90th percentile of the pooled per-seed final NashConv values of both arms — a level both arms reliably attain), the median time-to-band is fused 34.7 s vs sequential 190.2 s — ratio of the per-arm medians,  $5.48\times$  (these are time-to-band medians; the totals behind the  $9.99\times$  headline are the 76.8/767.2 s above). The non-monotone dip at  $\tau = 0.06$  ( $5.48\times$  between  $11.36\times$  and  $10.78\times$  neighbors) is a quantization artifact of the every-2-rounds measurement cadence — the fused arm first measures below this  $\tau$  at its round-3 measurement, the sequential arm at round 1; future runs will measure every round. We disclose the full range and keep the headline on the threshold-free total-wall number,  $9.99\times$ .
- **Substrate scope (best-vs-best).** The  $9.99\times$  is the same-substrate fused-vs-sequential comparison that isolates the mechanism (§3.3). Against the strongest *non-fused* cell in our own artifacts — the same kernel sequential on the host CPU — the per-belief  $2\times 2$  of §3.3 puts the G4 best-vs-best win at  $\sim 2.7\times$  (GPU-fused 0.201 s/belief vs CPU-sequential 0.547 s/belief), pending matched full-configuration CPU timings and the CUDA-Graphs/torch.compile sequential ablation, both planned (the CPU cells are single determinism-gate runs without warmup/repeat discipline; §3.3). In these runs, in the launch-bound regime, fusion is what made the GPU the faster substrate at all.
- **Gates.** CPU exact parity  $< 10^{-9}$ ; CPU determinism — fused and sequential training trajectories identical to all recorded digits (reduced  $4\times 10$  budget; §3.4); on CUDA, compounding `index_add` atomics produce per-seed fused-vs-sequential final-NashConv differences of median 0.0144 (max 0.0389) with no detectable bias at this seed count (fused higher on 2/5, lower on 3/5;  $n=5$  is underpowered to exclude moderate bias, and pooled with the quiet-host G5 pairs the tally is fused-higher on 4/8) — the §3.4 finding that per-seed equivalence checks of batched GPU CFR through a training loop will spuriously fail. Mild host-sensitivity is visible even in these clean G4 runs: the sequential arm’s per-seed inner totals spread 5.1% (749–788

End-to-end training wall per arm (G4, 5 seeds): total-wall speedup 9.99x

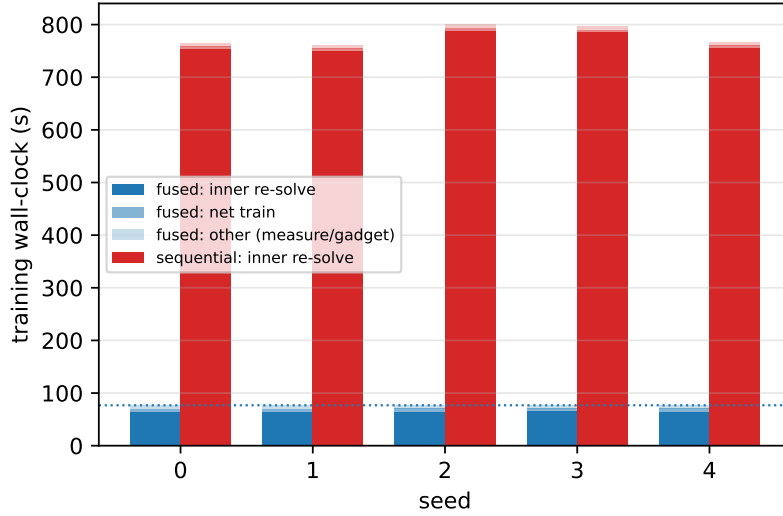


Figure 2: **The end-to-end win, per seed.** Training wall-clock per arm at Goofspiel-4, stacked by phase (inner re-solve / net train / other, where “other” is the separately-tracked measurement phase; the gadget pass runs after the timed loop and is outside all phase timers), for all 5 matched seeds. The inner re-solve is 98.4% of the sequential arm — the Amdahl fraction that lets the  $11.79\times$  inner speedup survive as a  $9.99\times$  total-wall speedup (dotted line: fused median). Source: `b0_e2e_g4_{fused,sequential}_seed{0..4}.json`.

s) vs the fused arm’s 3.5% — consistent with the §3.4 launch-loop host-load sensitivity at much smaller scale than the G5 event (§4.4).

### 4.3 Search-free boundary (directional only)

To calibrate what the resolving apparatus buys per unit compute — not to claim a converged comparison — we run NFSP in average-policy mode under one published configuration (§3.4; chosen over a tuned PPO-with-averaging baseline for the reasons given there [45]). After 600 s ( $\sim 640k/629k$  episodes on seeds 0/1), NFSP’s exact NashConv is **1.3968 / 1.3966**, barely below the uniform policy’s 1.4167; the fused resolving arm reaches its  $\sim 0.04$  final band in  $\sim 77$  s total — roughly 1/8th of that wall-clock, to a band NFSP has not approached. This is a *directional boundary only*: NFSP’s literature operating point is  $10^6$ – $10^7$  episodes, and we report it to locate the resolving loop on the compute axis, not to rank methods. The accompanying methodology point (§3.4): RPG/QPG current-policy NashConv oscillates at  $\approx 1.4334$  (recorded in the supplementary measurement notes, §7) — *above* uniform — so reporting it as the search-free boundary would artificially inflate our advantage, and we reject it; VRPO is cited as the strongest search-free boundary we are aware of and deliberately not reimplemented.

### 4.4 Goofspiel-5 at scale: the 5-seed band and the saturated-regime end-to-end point

**The 5-seed exploitability band.** Per-belief  $V^*$  targets — affordable at G5 precisely because the fused solver cuts the cost of the  $K=15$ -key population re-solve — robustly beat the **0.21**



Same band, no systematic bias  
(per-seed scatter = compounding CUDA atomics; CPU exact)

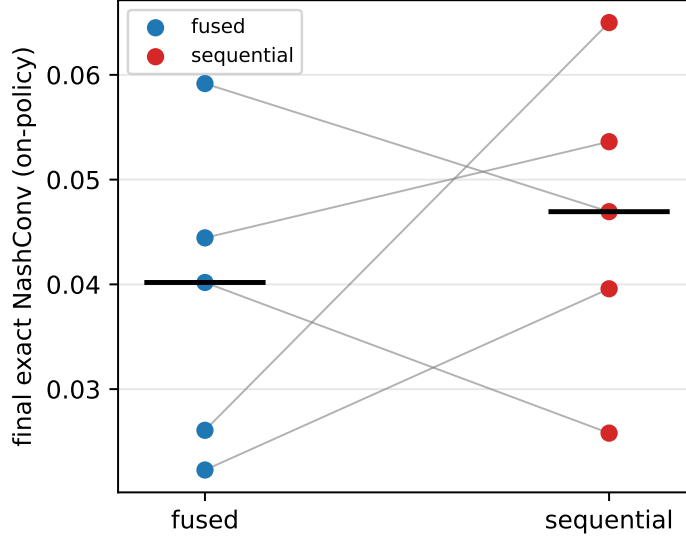


Figure 3: **Same band; no detectable bias** ( $n=5$ ). Final exact NashConv (on-policy continuation) of both arms at Goofspiel-4, per seed, with paired per-seed lines and medians (black bars): fused 0.0402 vs sequential 0.0469. The per-seed scatter (median  $|\text{diff}|$  0.0144, max 0.0389; fused higher on 2/5, lower on 3/5) is the compounding-CUDA-atomics effect of §3.4 — algorithmic equivalence is proven on CPU, where trajectories are identical to all recorded digits; the seed-sign tally is consistency evidence, not proof of unbiasedness. Source: `e2e_band_summary_g4.json` (per-seed values in `b0_e2e_g4_{fused,sequential}_seed{0..4}.json`).

**fixed-continuation floor.** The floor: in an earlier experiment under the same implementation and metric (artifacts in the release, §7), the value net was trained on targets computed under a *fixed* below-cut continuation policy rather than per-belief re-solving, and G5 exploitability plateaued at  $\approx 0.21$  regardless of additional net training — a *target-coherence* ceiling: the fixed continuation is incoherent off-path, so the net regresses toward values no assembled policy attains, bounding the loop independently of optimization effort (value-error-to-exploitability propagation in the sense of [26]). Under the identical advance-fixed protocol (§3.4 full configuration; 320 resolved beliefs per seed; exact NashConv on the assembled policy), the five seeds give:

- **On-policy band: median 0.1015 [10th/90th: 0.0825, 0.1222].** Gadget-safe band: median 0.1104 [10th/90th: 0.0932, 0.1319] — reported alongside per §3.4, with band claims on the on-policy continuation.
- **All 5/5 seeds finish below the 0.21 fixed-continuation floor;** the median improvement over the floor is **2.07 $\times$** . **Scope of this comparison (as stated at the start of §4):** this is a *target-construction* comparison — per-belief  $V^*$  targets vs fixed-continuation targets, i.e., what fusion makes affordable — and is orthogonal to fusion itself; it is **not** a fused-vs-sequential quality claim, and the arms comparison everywhere in this paper remains equal-band only.
- The final  $V^*$ -net validation MAE is tight across seeds (5.8–6.1%), and we find **no reproducible improves-with-target-compute trend below  $\sim 0.13$**  (across the 5 band seeds and the earlier seed-0/1 curve runs; §7) — initialization and optimization noise dominate there, which

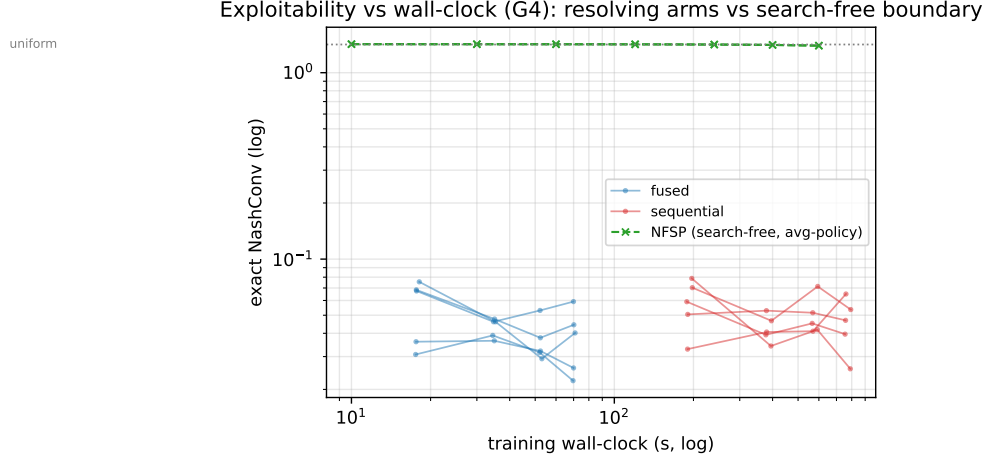


Figure 4: **Exploitability vs wall-clock (Goofspiel-4, log-log)**. Exact NashConv against cumulative training wall-clock: fused arm (blue) and sequential arm (red), 5 seeds each, with the NFSP average-policy curve (green, dashed) and the uniform-policy level (dotted). Both resolving arms descend to the same band — fused  $\sim 10\times$  earlier; NFSP remains near uniform for its full 600 s budget (directional boundary only). Source: `b0_e2e_g4*_seed*.json + searchfree_nfsp_g4_seed{0,1}.json`.

| seed | on-policy NashConv | gadget-safe NashConv | final $V^*$ -net val MAE |
|------|--------------------|----------------------|--------------------------|
| 0    | 0.0760             | 0.0880               | 5.8%                     |
| 1    | 0.1133             | 0.1228               | 6.1%                     |
| 2    | 0.1282             | 0.1380               | 6.1%                     |
| 3    | 0.0923             | 0.1010               | 6.0%                     |
| 4    | 0.1015             | 0.1104               | 6.1%                     |

Table 3: G5 5-seed per-belief  $V^*$  band (full 8-round protocol, seeds 0–4, fused path).

is why this is a band with percentile error bars and not a curve, and why all speed claims in this paper are equal-band-faster.

**The saturated-regime end-to-end point.** §4.1’s model predicts that at G5 — where a single subgame fills the device — the in-loop inner-resolve speedup should fall from G4’s  $\sim 11.7\times$  to the saturated plateau, microbenchmark value  $2.37\times$ . We test this with the same paired-arm design at G5 (shortened budget per §3.4: 4 rounds  $\times$  40 beliefs, single final exact measurement, gadget evaluation skipped). The headline numbers are the **quiet-host re-run: 3 matched seeds  $\times$  both arms on an otherwise idle machine, run after a host-load contamination diagnosis in the original 2-seed pair (disclosed below).**

- **Inner-resolve speedup  $2.40\times$  (median of per-seed ratios,  $n=3$ , quiet host; per-seed **2.373/2.400/2.398**)** — within  $\sim 1.1\%$  of the microbenchmark prediction of  $2.37\times$  (unrounded: 2.398 vs 2.372). Together with §4.2 (G4:  $\sim 11.7\times$  predicted  $\rightarrow 11.79\times$  measured,  $n=5$ ), the regime model’s microbenchmark-to-loop transfer holds at both ends.
- **The original 2-seed pair is superseded and retained as the motivating diagnostic** — its contamination reached the **GPU-event inner metric**. The diagnostic that

exposed it: the pair’s total-wall ratio ( $2.20\times$ ) *exceeded* the  $2.04\times$  Amdahl ceiling implied by its own measured inner fraction. Writing each arm’s total as inner plus non-inner time,  $T = I + O$ , with the per-seed definitions  $f = I_{\text{seq}}/T_{\text{seq}}$  and  $S_{\text{inner}} = I_{\text{seq}}/I_{\text{fus}}$ , gives  $T_{\text{seq}}/T_{\text{fus}} = 1/((1 - f) + f/S_{\text{inner}} + \Delta O/T_{\text{seq}})$  with  $\Delta O = O_{\text{fus}} - O_{\text{seq}}$ : a seed’s total-wall ratio exceeds its own implied ceiling only if its fused arm’s non-inner phases ran faster than the sequential arm’s ( $\Delta O < 0$ ) — and since the non-inner phases perform identical work in both arms (§3.4), any such excess can reflect only sub-percent timing noise (the quiet-host re-run itself realizes one such sub-percent per-seed excess, which this reading accommodates). The contaminated pair’s excess is an order of magnitude beyond that scale, which is what made it diagnostic (the same sequential seed-1 run also carries an anomalous 41.4 s net-train phase). The per-round artifacts then located the cause: seed 1’s *sequential* arm’s inner GPU-event times ran  $176.9 \rightarrow 237.1 / 206.3 / 206.5$  s (rounds 0–3), inflated 17–34% under concurrent host CPU load against seed 0’s flat baseline ( $177.8/177.1/177.2/177.1$  s), while the two *fused* arms were identical across seeds to 0.01%. This is the §3.4 mechanism: GPU-event spans bracketing a host-driven launch loop include host-stall gaps, so event timing does not immunize the launch-heavy sequential arm against host load — the fused arm, with  $\sim 15\times$  fewer launch sequences per belief, stays fed. Taken at face value, that pair’s  $n=2$  midpoint is  $2.57\times$ ; the contamination invalidates it, and restricting to the uncontaminated seed gives  $2.369\times$  ( $n=1$ ). The quiet-host  $n=3$  re-run supersedes both — prediction  $2.37\times$ , measurement  $2.40\times$  — and restores the self-consistency check the contaminated pair failed (next bullet).

- The inner re-solve is **86.1%** of the sequential loop (median,  $n=3$ , quiet host; below G4’s 98.4% because the single final measurement is a larger share of the shortened run).
- **Total-wall speedup  $2.00\times$  (median of per-seed ratios; per-seed 1.99/2.00/2.00), against the  $2.01\times$  Amdahl ceiling** implied by the measured 86.1% inner fraction and the  $2.40\times$  inner ratio — the accounting now closes to within 0.3% (computed from unrounded artifact values, as in §4.2), the same closure as G4’s, where the contaminated pair had violated its own ceiling. A time-to-band  $\tau$ -sweep is flat at  $2.39\times$  across  $\tau \in [0.05, 0.12]$ : with a single final measurement, time-to-band coincides with the training wall-clock, so the G4-style cadence sensitivity (§4.2) cannot arise here.
- **On final exploitability we make a divergence-scale-consistency statement, not a band claim.** Per-seed finals (fused, sequential): seed 0 (**0.1543, 0.0912**), seed 1 (**0.0974, 0.0857**), seed 2 (**0.1243, 0.1457**). The fused final is higher (worse) on **2/3** seeds — a direction  $n=3$  cannot assess, which we state rather than imply away. The per-seed divergences ( $0.0631/0.0117/0.0214$ ) are consistent in scale with the compounding-CUDA-atomics envelope documented at G4 (median 0.0144, max 0.0389; the G5 max exceeds it  $\sim 1.6\times$ ) — not inconsistent with the §3.4 mechanism, though the envelope’s scaling with game depth is uncharacterized, which the planned full-protocol G5 run will bound. These shortened-protocol (4-round) finals are *not comparable* to the 8-round 5-seed band above in either direction, and two of the six lie outside that band’s observed range — so no band-membership claim is available from these runs, and we make none. Through-training equivalence at G5 rests on the CPU gates and the single-solve parity ( $4 \times 10^{-6}$ ); establishing CUDA band equivalence at G5 requires the planned full-protocol  $n \geq 5$  paired run with a pre-registered margin. No lower-exploitability claim is made in either direction.
- The safe-resolving gadget is outside all timing claims: it runs after the timed loop, outside every phase timer and outside `total_wall` (§3.4). The quiet-host paired G5 runs skip it

uniformly; in the superseded original pair, the fused seed-0 run recorded one final gadget evaluation (0.1536) in that post-loop position — outside its timings and conservative with respect to the fused arm.

The two G5 results compose into the paper’s working point: the fused solver makes per-belief equilibrium targets affordable at 236k infosets (the band above), and even in the regime where fusion pays *least*, it still pays the predicted  $\sim 2.4\times$  on this device — with the prediction made by a model fit on microbenchmarks alone.

## 5 Limitations, Scope, and Future Work

**Scope of the demonstrated loop.** Our end-to-end demonstration is a **2-level, single-cut** iterated-resolving loop: the trunk is re-solved against value targets obtained by re-solving each cut subgame at sampled beliefs, and these per-belief targets are *independent given the belief*. This independence is exactly what makes the population of subgames fusable, so the obvious objection must be stated plainly: *is this just batching embarrassingly parallel work?* Our rebuttal is a conjunction, and we do not claim any single ingredient is novel in isolation: (a) identifying that train-time iterated resolving *structurally generates* a population of same-topology depth-limited PBS subgames — and that this maximally fusable shape is the lineage’s canonical target-generation workload (DeepStack’s  $10^7$  independently re-solved subgames, §3.1); (b) a fused CFR+ whose nontrivial content is the level-grouped compilation of irregular structure, float64 discipline through compounding fixed-point iterations, and live-loop integration, with parity to the sequential solver gated rather than assumed (§3.2 — the within-key coupled state is classical [22, 23] and not the novelty); (c) a predictive regime model, speedup  $\approx \min(N, 1/\varphi)$  with  $\varphi$  the subgame fill fraction, with an operationalized one-measurement regime probe (§4.1); and (d) an Amdahl-surviving end-to-end result (the ex-ante inner prediction  $11.7\times$  realized at  $11.79\times$ ; total-wall accounting closing to within 1% of the  $10.09\times$  ceiling). A relevant structural distinction for what lies beyond: DeepStack-style *independent target generation* is fully fusable, while ReBeL-style *episodic* resolving is serially coupled along an episode (fusable across parallel episodes and keys, not along the chain); and the belief axis is a second fusion dimension — our loop already solves 40 sampled beliefs per round as 40 separate fused solves over a belief-independent topology, so beliefs can be folded into the population to restore the fused width when level-coupling shrinks  $K$ . The strongest form of the claim — fused-vs-sequential **timing under  $\geq 3$ -level nested resolving**, where targets become serially coupled across levels — is deferred to future work. Multi-level self-play *correctness* on this implementation is already certified (Goofspiel-4: exploitability decreases  $0.0506 \rightarrow 0.0145$  with training; recorded in the supplementary measurement notes, §7; a standalone artifact is planned); only the multi-level timing comparison is outstanding.

**Baseline tooling and substrate.** The launch-bound speedups are measured against an eager-dispatch, single-stream sequential arm. Per §3.3, a CUDA-Graphs-captured or `torch.compile(reduce-overhead)` sequential arm is plausibly buildable for this kernel (static shapes per topology; the obstacles are iteration-indexed scalars and out-of-place state updates, both engineering-level), and our own CPU-gate artifacts imply that at G4 scale the GPU-sequential arm is  $\sim 4.3\times$  slower per belief than the same kernel on the host CPU — making the cross-substrate best-vs-best G4 win  $\sim 2.7\times$  as currently measurable (single-run CPU cells, no timing discipline; §3.3). Both the graphs/compile ablation and matched full-configuration CPU timings (G4 and G5) are planned; until they are complete, the  $\sim 10\times$  figure is a same-substrate, eager-tooling-class comparison, and we state it as such everywhere it appears. Multi-stream per-tree dispatch and fp32-with-compensated-accumulation are further untested alternatives in the same family.

**One game family, one device.** All end-to-end timing is on the Goofspiel ladder; Leduc and liar’s-dice are parity-tested for the fused solver but not end-to-end timed. All measurements use a single consumer GPU (RTX 3070 Ti, 8 GB) whose FP64 throughput is 1:64 of FP32 — an extreme placement for a float64 workload. The regime model is likewise subject to memory capacity (§4.1): the 8 GB card bounds the fusable population ( $B_{\max}$  unmeasured; every population in this paper fits), and peak VRAM per arm is not yet instrumented — a named reporting blind spot pending the planned instrumented runs. The direction of this asymmetry is *favorable* to the lever: on a data-center part with 1:2 FP64 (A100) and  $\sim 10\times$  the FP64 throughput, G5-sized subgames would sit much further from arithmetic saturation and likely re-enter the launch-bound regime — bigger fused wins on bigger games — while cheaper effective launches would shrink the small-game gap; both regime parameters move, the model’s form does not. On this device, the both-ends validation and the HUNL-mass argument below hold as measured; their numeric values are device-conditional, and A100 replication is future work.

**Exploitability is a band, not a curve.** On the largest instance (Goofspiel-5, 236,450 infosets) final exploitability sits in a 0.07–0.13 band where initialization and optimization noise dominate (below  $\sim 0.13$  we found no reproducible improves-with-target-compute trend). We therefore make **time-based, equal-band claims only** — never lower-exploitability claims — with both arms sharing seeds. The completed 5-seed G5 band is on-policy median 0.1015 [10th/90th: 0.0825, 0.1222] (gadget-safe median 0.1104 [0.0932, 0.1319]; all 5/5 below the 0.21 fixed-continuation floor, defined in §4.4). The G5 paired end-to-end finals are shortened-protocol values that are not band-comparable and have fused higher on 2/3 seeds (quiet-host re-run); per §4.4 we make a divergence-scale-consistency statement there, not a band claim. Fused-vs-sequential equivalence is proven on CPU (exact parity  $< 10^{-9}$ ; determinism identical to all recorded digits at the reduced gate budget); on CUDA, `index_add_` atomics compound through training into per-seed final-exploitability differences of median 0.0144 (max 0.0389) at G4 with no detectable bias at  $n=5$  (§3.4); the divergence envelope’s scaling with game depth is uncharacterized — the G5 max divergence exceeds the G4 max (§4.4) — and the planned full-protocol G5 run will bound it. Gadget-based safe-resolving numbers additionally inherit refinement-choice sensitivity [29].

**The saturated regime is a bound, and we state it as one.** When a single subgame occupies the device, fusion degrades to  $\sim 2.4\times$  on this device (Goofspiel-5: microbenchmark  $2.37\times$ , measured in the end-to-end loop at  $2.40\times$ ,  $n=3$  quiet host, §4.4). We regard this as a *contribution* — the cost model predicts where the lever pays (the launch-bound, many-small-subgames regime that iterated resolving structurally generates) and where it does not — but it caps the claim: the  $\sim 10\times$  figure is regime-conditional, not universal. The measured inner fractions are likewise operating-point properties: the 98.4% (G4) and 86.1% (G5) inner shares reflect a small value network (MLP, hidden 128) and a sparse measurement cadence; with production-scale value networks the non-inner share grows,  $f$  drops, and the total-wall multiple contracts toward the inner-resolve ratio applied to a smaller fraction — exactly as the Amdahl decomposition prescribes, and in the direction the G5 point already shows ( $f = 86.1\%$  gives a  $2.01\times$  ceiling).

**Subgame-build ceiling.** The fused *solver* scales ( $\approx 6.2$  ms/iter in-loop at Goofspiel-5), but our eager full-enumeration subgame *build* does not: Goofspiel-6 build exceeds 280 s, capping the demonstrated ladder at  $\sim 236$ k infosets — short of the  $10^6$ – $10^7$  infosets of the large games evaluated in [45]. Lazy/direct-SoA construction is future work, not a solver limitation.

**Future work.** (1) The  $\geq 3$ -level fused-vs-sequential timing demonstration. (2) The planned baseline ablations: CUDA-Graphs/`torch.compile` sequential arm and matched CPU timings; the quiet-host G5 timing re-run is complete ( $n=3$ , §4.4), and what remains on that axis is the full-protocol (8-round)  $n \geq 5$  paired run with a pre-registered band margin for a CUDA band-equivalence test. (3) A second game family timed end-to-end. (4) A100 replication and multi-GPU sharding —

the population axis shards trivially across devices (unlike a single tree), so multi-GPU *extends* the lever rather than competing with it. (5) A HUNL-scale instance built on lazy subgame construction, where held-out Slumbot [21] becomes the transfer validation; consistent with our protocol, Slumbot supplies no training data, targets, or selection signal at any point. A preliminary measured probe on real HUNL census topology — out of scope for this paper’s claims, reported only as a forward pointer — supports the transfer on this device (median of 3 repeats per point, one seed): in a census of the HUNL public betting tree under our 9-action abstraction, the dominant light river trees (median 21 nodes; 95.9% of river configurations fall in topology classes with  $\geq 16$  members) are still **launch-bound** ( $m = 0.062$  on the §4.1 probe), with fused-vs-sequential  $10.86\times$  at  $K=16$  co-solved trees and  $9.45\times$  at  $K=154$  — the speedup recedes slightly at  $K=154$ , a decline outside the min form’s monotone envelope, consistent with memory-traffic effects beyond launch amortization and a known limit of the zero-overhead envelope (§4.1) — while the heavy 95th-percentile trees (411 nodes) saturate ( $m = 0.278$ ) at  $3.35\times$  ( $K=16$ ) /  $3.74\times$  ( $K=51$ ). Two probe caveats: the probe’s range dimension is 1,081 private hands per player (the river hand set under our abstraction), with synthetic shape-faithful payoff matrices (timing-only, no Slumbot interaction); and a class-size census is not a per-sweep co-occurrence count — the batch one can actually form at train time is the number of same-topology subgames co-occurring in one resolve sweep, which the census upper-bounds but does not measure. (6) Composition with the other levers of Eq. (1): pairing the population axis with TurboReBeL-style  $N_{\text{solves}}$  reduction, DCFR/PCFR+/meta-learned iteration reduction, or MCCFR-family variance levers is untested — including the interaction risk that predictive or meta-learned minimizers change the per-iteration op mix the compiled kernel assumes. The within-loop sequential arm — the same kernel run per-tree [24] — remains the load-bearing baseline in every experiment.

## 6 Conclusion

Train-time iterated resolving structurally generates populations of same-topology depth-limited PBS subgames, and fusing each population into one level-grouped SoA CFR+ claims the unclaimed across-solve amortization of the cost-per-solve factor of its cost identity — wall-clock per solve-iteration, hence, at the common iteration budget  $T$ , per-solve wall-clock — with parity to per-tree solving gated at float64 and, on CPU, trajectory-identical to all recorded digits. The claim is deliberately narrow: *equal-band-faster*, same-substrate against the same kernel run per tree, in a regime the cost model identifies and a one-measurement probe classifies — near-linear in the number of co-solved subgames while solves are launch-bound (Goofspiel-4: inner  $11.79\times$  predicted ex ante at  $11.7\times$ ; total wall  $9.99\times$  against a  $10.09\times$  Amdahl ceiling, 5 matched seeds), saturating at  $\sim 2.4\times$  when one subgame fills the device (Goofspiel-5:  $2.37\times$  predicted,  $2.40\times$  measured, 3 matched quiet-host seeds; equal-band certified on CPU, the CUDA band test pending). We claim no exploitability improvement, no cross-substrate superiority beyond the  $\sim 2.7\times$  best-vs-best our artifacts currently support at G4 (single-run CPU cells; §3.3), and no generality beyond one game family and one device until the planned ablations and replications are complete. What we believe travels regardless: the population-fusion lever and its regime model, the matched-seed paired-arm + CPU-exactness-gate evaluation template, and the warning that compounding CUDA atomics make naive per-seed equivalence checks of batched GPU CFR spuriously fail.



## 7 Reproducibility Statement

**Code and commits.** All experiments live in one repository (release accompanies the preprint). The evidence-bearing commits:

- c47cc19 — cost-per-solve microbenchmark: fused population solve vs sequential per-tree GPU-CFR (controlled key sweep, exact-parity check). Produces `costpersolve_bakeoff.json`.
- 8def574 — end-to-end train-time win (G4 paired arms, 5 matched seeds) + NFSP search-free boundary. Produces `b0_e2e_g4_{fused,sequential}_seed{0..4}.json`, `e2e_band_summary_g4.json`, `searchfree_nfsp_g4_seed{0,1}.json`.
- 1ff6a8e — G5 per-belief  $V^*$  exploitability band, seeds 0–1. Produces `b0_perbelief_g5_curve.json`, `b0_perbelief_g5_curve_seed1.json`.
- Final-data runs (post dd3d148; release commit pinned at submission) — G5 band seeds 2–4 (`b0_perbelief_g5_curve_seed{2..4}.json`), the original G5 paired end-to-end pair (`b0_e2e_g5_{fused,sequential}_seed{0,1}.json`, `e2e_band_summary_g5.json`; superseded for headline use, retained as the §4.4 contamination diagnostic), the quiet-host G5 paired re-run (`b0_e2e_g5q_{fused,sequential}_seed{0,1,2}.json`, `e2e_band_summary_g5q.json`; the §4.4 headline), and the HUNL topology census + GPU timing probe (`hunl_topology_census_probe.json`).

**Artifacts accompany the paper.** Every JSON cited in this paper is copied into the tracked directory `release/v1-artifacts/` with a `README.md` manifest and per-file SHA-256 sums (the copies under `release/v1-artifacts/` are the citable artifacts and accompany the release/DOI deposit; the raw run outputs under `autoresearch-session/` are not tracked). This includes the CPU determinism-gate artifacts (`b0_cpucheck_g4_{fused,sequential}.json`) behind the §3.3  $2 \times 2$ , and a supplementary measurement-notes file (`supplementary_measurement_notes.md`) documenting the five numbers not yet backed by a dedicated JSON (the  $7 \times 10^{-6}$  cross-game parity upper end; RPG/QPG 1.4334; multi-level 0.0506→0.0145; G6 build >280 s; the ~0.75 L1 float32-ablation drift of §3.2), which will be re-derived as standalone artifacts in a future version of the artifact release; the in-loop G5 per-iteration figure ( $\approx 6.2$  ms, §3.4) is derived from the released end-to-end artifacts (per-round inner-resolve wall-clock over 40 beliefs  $\times$  300 iterations per solve). The G5 topology census of §3.4 (26,773 nodes across 10 level groups,  $K=15$ ,  $B=125$ ; 236,450 total info-sets, of which 236,440 are below-cut rows in the batched tables) is not in that notes file; it is re-derived directly from the compiled topology — a single call to `_compile_topology` on G5 — and the derivation line is recorded in the `release/v1-artifacts/README.md`. Field-naming note: the `reach_{fused,sequential}_s_median` fields in `e2e_band_summary_g*.json` are **time-to-band medians at  $\tau$  = the pooled band ceiling** (§4.2), not protocol totals; the field names are retained for artifact stability and documented here and in the artifact `README`.

**One-command repro lines** (defaults encode the single advance-fixed configuration of §3.4; raw JSONs are written under `autoresearch-session/rebel/`):

```
# 4.1 microbenchmark + controlled key sweep (3 warmups discarded, median of 7)
python scripts/run_rebel_costpersolve_bakeoff.py --num-cards 3 4 5 --iters 300 \
  --repeats 7 --warmup 3 --key-sweep 4 5 \
  --output-json autoresearch-session/rebel/costpersolve_bakeoff.json

# 4.2 end-to-end paired arms (per seed s in 0..4; both arms share the seed)
for s in 0 1 2 3 4; do for m in fused sequential; do
```

```

python scripts/run_rebel_b0_perbelief_targets.py --num-cards 4 --seed $s \
    --inner-mode $m \
    --output-json autoresearch-session/rebel/b0_e2e_g4_${m}_seed${s}.json
done; done

# 4.2 aggregation: medians of per-seed ratios + band summary
python scripts/analyze_rebel_e2e_band.py \
    --glob 'autoresearch-session/rebel/b0_e2e_g4_*_seed*.json' \
    --output-json autoresearch-session/rebel/e2e_band_summary_g4.json

# 4.3 NFSP search-free boundary (seeds 0 and 1)
python scripts/run_rebel_searchfree_baseline.py --num-cards 4 --method nfsp \
    --seed 0 --max-seconds 600 \
    --output-json autoresearch-session/rebel/searchfree_nfsp_g4_seed0.json

# 4.4 G5 5-seed exploitability band (full 8-round configuration; seeds 0..4)
for s in 0 1 2 3 4; do
    python scripts/run_rebel_b0_perbelief_targets.py --num-cards 5 --seed $s \
        --output-json autoresearch-session/rebel/b0_perbelief_g5_curve_seed${s}.json
done # seeds 0-1 were produced earlier as b0_perbelief_g5_curve{,_seed1}.json

# 4.4 G5 paired end-to-end (saturated regime; shortened 4-round protocol, gadget skipped).
# Headline = quiet-host re-run, seeds 0..2, machine otherwise idle; the original
# seeds-{0,1} pair (b0_e2e_g5_* + e2e_band_summary_g5.json) is superseded and
# retained as the 4.4 contamination diagnostic.
for s in 0 1 2; do for m in fused sequential; do
    python scripts/run_rebel_b0_perbelief_targets.py --num-cards 5 --seed $s \
        --inner-mode $m --n-target-rounds 4 --measure-every 4 --skip-gadget \
        --output-json autoresearch-session/rebel/b0_e2e_g5q_${m}_seed${s}.json
done; done
python scripts/analyze_rebel_e2e_band.py \
    --glob 'autoresearch-session/rebel/b0_e2e_g5q_*_seed*.json' \
    --output-json autoresearch-session/rebel/e2e_band_summary_g5q.json

# 5 HUNL topology census + GPU timing probe (forward pointer; out of scope)
python scripts/estimate_hunl_topology_buckets.py --gpu-probe \
    --output autoresearch-session/rebel/hunl_topology_census_probe.json

# Figures 1-4 (pure read of the JSONs above)
python scripts/make_rebel_e2e_figures.py

# Exactness gates (CPU). The fused entry point under test is solve_all_keys_soa
# (poker_ai/rebel/iig_batched.py).
TESTING_SUITE=1 pytest \
    test/unit/test_rebel_iig_batched.py::test_fused_vs_sequential_soa_parity -v
# CPU determinism gate (reduced budget, disclosed in 3.4): run the 4.2 command pair
# with --device cpu --n-target-rounds 4 --samples-per-round 10 --subgame-iters 300
# (the last flag restates the default, pinning the 3.3 per-belief iteration count);
# trajectories are identical to all recorded digits between --inner-mode fused and
# sequential (artifacts: b0_cpucheck_g4_{fused,sequential}.json -- also the source of
# the 3.3 per-belief CPU timing cells).
```

**Seeds.** G4 end-to-end: seeds 0–4, matched pairs (same seed drives both arms). G5 end-to-end: quiet-host re-run seeds 0–2, matched pairs (the headline); the original seeds 0–1 pair is superseded

(seed 1’s sequential arm contaminated by host load) and retained as the §4.4 diagnostic. NFSP: seeds 0–1. G5 band: seeds 0–4. HUNL GPU probe: seed 0. No other seeds were run; no configuration sweeps were performed (single configuration fixed in advance; §3.4; the G5 end-to-end budget shortening is disclosed there and in §4.4).

**Aggregation and timing.** Headline speedups follow the §3.4 median-of-per-seed-ratios convention; the microbenchmark and time-to-band speedups are the ratio-of-medians exceptions stated there (per-seed values reported directly when  $n \leq 2$ ). GPU timing uses `torch.cuda.Event` pairs with `torch.cuda.synchronize()` barriers — with the §3.4 caveat that event spans bracketing host-driven launch loops are host-load-sensitive; microbenchmarks discard 3 warmup runs and report the median of 7 repeats on precompiled topologies.

**Hardware/software.** GPU: one NVIDIA GeForce RTX 3070 Ti (8 GB, FP64 1:64), driver 595.71, PyTorch float64 tensor ops. Host: Intel Core i7-12700K (12 cores / 20 threads), 32 GB RAM, Ubuntu 24.04 LTS. Software: Python 3.13, PyTorch 2.12 (CUDA 13.0 build), OpenSpiel 1.6.15. Exactness gates run the identical code on the host CPU; NFSP runs CPU-only (§3.4). Total compute: the artifact-recorded GPU runs of this paper — both arms, all seeds, including the superseded contaminated G5 pair and the G5 band seeds 2–4 — sum to  $\approx 4.3$  GPU-hours on the single device; the two earliest G5 band runs (seeds 0–1) predate the wall-clock field and are of comparable scale to the later band seeds, and the §4.1 microbenchmark plus the HUNL probe add minutes. Exploitability is exact NashConv via OpenSpiel’s best response [32] — no sampled exploitability anywhere.

**Held-out integrity.** No Slumbot (or any external-agent) data, targets, priors, or selection signals are used anywhere in this paper (`uses_slumbot_data=false`). The HUNL topology census probe (§5) uses only the public betting-tree structure under our own abstraction and synthetic shape-faithful payoff matrices for timing — no Slumbot interaction of any kind.

## References

- [1] A. Abdelfattah, A. Haidar, S. Tomov, J. Dongarra. Performance, Design, and Autotuning of Batched GEMM for GPUs. *ISC High Performance* 2016.
- [2] A. Abdelfattah, T. Costa, J. Dongarra, et al. A Set of Batched Basic Linear Algebra Subprograms and LAPACK Routines. *ACM TOMS* 47(3), 2021 (the Batched BLAS standard).
- [3] G. M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *AFIPS Spring Joint Computer Conference* 1967.
- [4] S. Baghal. Solving Pasur Using GPU-Accelerated Counterfactual Regret Minimization. arXiv:2508.06559, 2025.
- [5] M. Bowling, N. Burch, M. Johanson, O. Tammelin. Heads-up limit hold’em poker is solved. *Science* 347(6218):145–149, 2015.
- [6] N. Brown, T. Sandholm. Safe and Nested Subgame Solving for Imperfect-Information Games. *NeurIPS* 2017.
- [7] N. Brown, T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* 359(6374):418–424, 2018 (published online December 2017).
- [8] N. Brown, T. Sandholm, B. Amos. Depth-Limited Solving for Imperfect-Information Games. *NeurIPS* 2018 (arXiv:1805.08195).

- [9] N. Brown, T. Sandholm. Solving Imperfect-Information Games via Discounted Regret Minimization. *AAAI* 2019 (DCFR).
- [10] N. Brown, A. Lerer, S. Gross, T. Sandholm. Deep Counterfactual Regret Minimization. *ICML* 2019 (arXiv:1811.00164).
- [11] N. Brown, T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365(6456):885–890, 2019.
- [12] N. Brown, A. Bakhtin, A. Lerer, Q. Gong. Combining Deep Reinforcement Learning and Search for Imperfect-Information Games (ReBeL). *NeurIPS* 2020 (arXiv:2007.13544).
- [13] N. Burch, M. Johanson, M. Bowling. Solving Imperfect Information Games Using Decomposition. *AAAI* 2014.
- [14] N. Burch, M. Moravčík, M. Schmid. Revisiting CFR+ and Alternating Updates. *Journal of Artificial Intelligence Research* 64:429–443, 2019.
- [15] I. Danihelka, A. Guez, J. Schrittwieser, D. Silver. Policy Improvement by Planning with Gumbel. *ICLR* 2022 (implemented in the mctx library, [github.com/google-deepmind/mctx](https://github.com/google-deepmind/mctx)).
- [16] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS* 2022.
- [17] Z. Fan, G. Farina. GAE Falls Short in Imperfect-Information Self-Play Reinforcement Learning. arXiv:2605.19235, 2026 (introduces VRPO).
- [18] G. Farina, C. Kroer, T. Sandholm. Faster Game Solving via Predictive Blackwell Approachability: Connecting Regret Matching and Mirror Descent. *AAAI* 2021 (PCFR+).
- [19] S. Hart, A. Mas-Colell. A Simple Adaptive Procedure Leading to Correlated Equilibrium. *Econometrica* 68(5):1127–1150, 2000.
- [20] J. Heinrich, D. Silver. Deep Reinforcement Learning from Self-Play in Imperfect-Information Games. arXiv:1603.01121, 2016.
- [21] E. G. Jackson. Slumbot NL: Solving Large Games with Counterfactual Regret Minimization Using Sampling and Distributed Processing. *AAAI Workshop on Computer Poker and Imperfect Information*, 2013.
- [22] M. Johanson, K. Waugh, M. Bowling, M. Zinkevich. Accelerating Best Response Calculation in Large Extensive Games. *IJCAI* 2011.
- [23] M. Johanson, N. Bard, M. Lanctot, R. Gibson, M. Bowling. Efficient Nash Equilibrium Approximation through Monte Carlo Counterfactual Regret Minimization. *AAMAS* 2012.
- [24] J. Kim. GPU-Accelerated Counterfactual Regret Minimization. arXiv:2408.14778, 2024.
- [25] J. Kim, T. Sandholm. Parallelizing Counterfactual Regret Minimization. arXiv:2605.14277, 2026 (the authors’ NoRegret library, v0.0.0.dev15, 2026-06-04).
- [26] V. Kovařík, D. Seitz, V. Lisý, J. Rudolf, S. Sun, K. Ha. Value Functions for Depth-Limited Solving in Zero-Sum Imperfect-Information Games. *Artificial Intelligence* 314:103805, 2023 (arXiv:1906.06412).

- [27] S. Koyamada et al. Pgx: Hardware-Accelerated Parallel Game Simulators for Reinforcement Learning. *NeurIPS 2023* (Datasets and Benchmarks track).
- [28] O. Kubíček, V. Lisý. Look-ahead Reasoning with a Learned Model in Imperfect Information Games. *ICLR 2026* (poster; arXiv:2510.05048). (Introduces the LAMIR algorithm.)
- [29] O. Kubíček, V. Lisý, T. Sandholm. Equilibrium Refinements Improve Subgame Solving in Imperfect-Information Games. arXiv:2601.17131, 2026.
- [30] M. Lanctot, K. Waugh, M. Zinkevich, M. Bowling. Monte Carlo Sampling for Regret Minimization in Extensive Games. *NeurIPS 2009*.
- [31] M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver, T. Graepel. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning. *NeurIPS 2017* (arXiv:1711.00832).
- [32] M. Lanctot et al. OpenSpiel: A Framework for Reinforcement Learning in Games. arXiv:1908.09453, 2019.
- [33] H. Li, L. Song (Alipay Hangzhou Information Technology Co.). Determining action selection policies of an execution device. US Patent 11,204,803 B2, granted Dec. 21, 2021 (decomposes large imperfect-information games into independently solvable subtasks).
- [34] B. Li, L. Huang. TurboReBeL:  $250\times$  Accelerated Belief Learning for large Imperfect-Information Extensive-Form Games. OpenReview preprint yMo7Z670f6, 2025.
- [35] Z. Li, J. Schultz, D. Hennes, M. Lanctot. Discovering Multiagent Learning Algorithms with Large Language Models. arXiv:2602.16928, 2026.
- [36] V. Makoviyshuk et al. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. *NeurIPS 2021* (Datasets and Benchmarks track; arXiv:2108.10470).
- [37] L. Marris et al. Turbocharging Solution Concepts: Solving NEs, CEs and CCEs with Neural Equilibrium Solvers. *NeurIPS 2022* (arXiv:2210.09257).
- [38] S. McAleer, G. Farina, M. Lanctot, T. Sandholm. ESCHER: Eschewing Importance Sampling in Games by Computing a History Value Function to Estimate Regret. *ICLR 2023*.
- [39] D. Milec, O. Kubíček, V. Lisý. Continual Depth-limited Responses for Computing Counter-strategies in Sequential Games. arXiv:2112.12594, 2021.
- [40] M. Moravčík, M. Schmid, K. Ha, M. Hladík, S. Gaukrodger. Refining Subgames in Large Imperfect Information Games. *AAAI 2016*.
- [41] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, M. Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science* 356(6337):508–513, 2017 (arXiv:1701.01724).
- [42] NVIDIA. CUDA Graphs. *CUDA C++ Programming Guide* (feature introduced CUDA 10, 2018); A. Gray, Getting Started with CUDA Graphs, NVIDIA Developer Blog, 2019.
- [43] S. M. Ross. Goofspiel — The Game of Pure Strategy. *Journal of Applied Probability* 8(3):621–625, 1971.

- [44] J. Rudolf. GPUCFR: a parallel CFR implementation in C++/CUDA. Code, CTU Prague, [github.com/janrvdolf/gpucfr](https://github.com/janrvdolf/gpucfr).
- [45] M. Rudolph et al. Reevaluating Policy Gradient Methods for Imperfect-Information Games. *ICLR* 2026 (arXiv:2502.08938).
- [46] M. Schmid, N. Burch, M. Lanctot, M. Moravčík, R. Kadlec, M. Bowling. Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines. *AAAI* 2019.
- [47] M. Schmid et al. Student of Games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances* 9(46):eadg3256, 2023 (preprint “Player of Games,” arXiv:2112.03178).
- [48] S. Srinivasan, M. Lanctot, V. Zambaldi, J. Pérolat, K. Tuyls, R. Munos, M. Bowling. Actor-Critic Policy Optimization in Partially Observable Multiagent Environments. *NeurIPS* 2018 (arXiv:1810.09026).
- [49] E. Steinberger, A. Lerer, N. Brown. DREAM: Deep Regret Minimization with Advantage Baselines and Model-free Learning. arXiv:2006.10410, 2020.
- [50] D. Sychrovský, M. Šustr, E. Davoodi, M. Bowling, M. Lanctot, M. Schmid. Learning not to Regret. *AAAI* 2024 (arXiv:2303.01074).
- [51] D. Sychrovský, M. Schmid, M. Šustr, M. Bowling. Meta-Learning in Self-Play Regret Minimization. arXiv:2504.18917, 2025.
- [52] O. Tammelin. Solving Large Imperfect Information Games Using CFR+. arXiv:1407.5042, 2014.
- [53] O. Tammelin, N. Burch, M. Johanson, M. Bowling. Solving Heads-Up Limit Texas Hold’em. *IJCAI* 2015.
- [54] J. Weng et al. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. *NeurIPS* 2022 (Datasets and Benchmarks track).
- [55] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *CACM* 52(4):65–76, 2009.
- [56] H. Xu, K. Li, H. Fu, Q. Fu, J. Xing, J. Cheng. Deep (Predictive) Discounted Counterfactual Regret Minimization. *AAAI* 2026 (arXiv:2511.08174).
- [57] M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret Minimization in Games with Incomplete Information. *NeurIPS* 2007.